

BAB IV

HASIL DAN PEMBAHASAN

Bab ini membahas hasil implementasi dari sistem informasi manajemen sidang skripsi yang telah dirancang pada bab sebelumnya. Pembahasan meliputi implementasi sistem, implementasi antarmuka berdasarkan peran pengguna, serta penjelasan fitur-fitur utama yang tersedia pada sistem. Selain itu, bab ini juga menyajikan hasil pengujian menggunakan System Usability Scale (SUS) untuk mengetahui tingkat kemudahan penggunaan dan penerimaan pengguna terhadap sistem yang dikembangkan.

4.1 Implementasi Sistem

Implementasi sistem merupakan tahap penerapan dari hasil analisis dan perancangan yang telah dijelaskan pada bab sebelumnya. Pada tahap ini, seluruh rancangan sistem direalisasikan ke dalam bentuk *website* SIMPRO yang dapat digunakan oleh pengguna sesuai dengan perannya masing-masing.

Sistem ini dikembangkan sebagai *website* dengan menggunakan *framework* Next.js sebagai teknologi utama pada sisi antarmuka (*frontend*). Next.js dipilih karena mendukung pengembangan aplikasi *web* modern yang bersifat modular, responsif, serta mampu menangani proses rendering secara efisien. Untuk mendukung tampilan antarmuka, sistem ini menggunakan *Tailwind CSS* yang mempermudah pembuatan desain antarmuka yang konsisten dan responsif

Pada sisi pengelolaan data, sistem ini memanfaatkan Supabase sebagai basis data sekaligus layanan *backend*. Supabase digunakan untuk mengelola data pengguna, proposal skripsi, bimbingan, dokumen seminar, penjadwalan seminar dan sidang, serta relasi antar data yang tersimpan dalam basis data PostgreSQL. Selain itu, Supabase juga digunakan untuk menangani proses autentikasi pengguna serta penyimpanan dokumen yang diunggah oleh mahasiswa.

Arsitektur sistem yang dibangun menerapkan konsep *client side application*, di mana aplikasi *frontend* berkomunikasi langsung dengan layanan Supabase melalui *Application Programming Interface* (API). Pola arsitektur ini memungkinkan sistem berjalan secara efisien tanpa memerlukan *server backend* terpisah, sehingga mempermudah proses pengembangan dan pemeliharaan sistem.

Proses pengembangan sistem dilakukan pada lingkungan lokal (*localhost*), sedangkan tahap pengujian dan implementasi *website* dilakukan melalui platform Vercel sebagai media deployment berbasis *cloud*. Sistem diakses menggunakan peramban *web*, sehingga dapat digunakan secara daring. Dengan implementasi tersebut, sistem yang dibangun mampu mendukung seluruh kebutuhan utama dalam pengelolaan sidang skripsi, mulai dari pengajuan proposal, proses bimbingan, verifikasi dokumen, hingga penjadwalan seminar dan sidang mahasiswa.

4.2 Implementasi Antarmuka

Implementasi antarmuka merupakan tahap penerapan rancangan antarmuka pengguna yang telah dibuat pada tahap perancangan sistem ke dalam bentuk *website* yang dapat digunakan secara langsung oleh pengguna. Antarmuka pada

sistem ini dirancang dengan tujuan untuk memberikan kemudahan dalam penggunaan (*user friendly*), kejelasan informasi, serta konsistensi tampilan pada setiap halaman.

Antarmuka sistem dikembangkan berbasis *website* dan disesuaikan dengan kebutuhan masing-masing peran pengguna, yaitu mahasiswa, dosen, ketua program studi, dan tenaga kependidikan. Setiap peran memiliki tampilan dan fitur yang berbeda sesuai dengan hak akses dan fungsinya dalam sistem. Hal ini bertujuan agar pengguna dapat mengakses informasi dan melakukan aktivitas yang diperlukan secara efektif dan efisien.

Pada bagian ini akan ditampilkan hasil implementasi antarmuka dari halaman-halaman utama pada sistem, mulai dari halaman login hingga halaman verifikasi kelulusan. Setiap halaman akan disertai dengan penjelasan singkat mengenai fungsi, informasi yang ditampilkan, serta pemaparan potongan kode utama (*source code*) yang bertugas membangun fungsionalitas dan logika pada halaman tersebut.

4.2.1 Halaman *Signup* dan *Login*

Halaman *signup* dan *login* merupakan bagian awal dari antarmuka Sistem Informasi Manajemen Sidang Skripsi (SIMPRO) yang berfungsi sebagai mekanisme autentikasi pengguna. Melalui halaman ini, sistem memastikan bahwa hanya pengguna yang terdaftar dan memiliki hak akses yang sesuai yang dapat menggunakan fitur-fitur yang tersedia di dalam sistem.

Halaman *signup* digunakan oleh pengguna yang belum memiliki akun untuk melakukan pendaftaran, sedangkan halaman *login* digunakan oleh pengguna yang

telah terdaftar untuk masuk ke dalam sistem. Proses autentikasi pada sistem ini dirancang agar mudah digunakan, cepat, dan aman, sehingga pengguna dapat mengakses sistem secara efisien sebelum diarahkan ke halaman dashboard sesuai dengan peran masing-masing.



Gambar 4.1 (a) Halaman sign up dan (b) Halaman login

Pada Gambar 4.1(a) ditampilkan halaman pendaftaran akun. Halaman ini digunakan oleh pengguna yang belum memiliki akun untuk melakukan registrasi ke dalam sistem. Proses pendaftaran dilakukan dengan menggunakan akun *Google* melalui tombol ‘Daftar dengan *Google*’, sehingga pengguna tidak perlu mengisi data secara manual. Pendekatan ini bertujuan untuk mempermudah proses pendaftaran serta mengurangi kesalahan pengisian data.

Sementara itu, Gambar 4.1(b) menampilkan halaman *login* yang digunakan oleh pengguna yang telah memiliki akun untuk masuk ke dalam sistem. Proses *login*

juga dilakukan melalui akun *Google* dengan menekan tombol ‘Masuk dengan *Google*’. Setelah proses autentikasi berhasil, pengguna akan diarahkan ke halaman *dashboard* sesuai dengan peran masing-masing, yaitu mahasiswa, dosen, kaprodi atau tendik.

Sistem menyediakan proses autentikasi menggunakan *Google OAuth* melalui *Supabase Authentication* sehingga pengguna dapat melakukan registrasi maupun *login* menggunakan akun *Google*. Potongan kode berikut digunakan untuk melakukan proses pendaftaran akun menggunakan *Google OAuth*.

```
await supabase.auth
  .signInWithOAuth({
    provider: "google",
    options: {
      redirectTo:
        `${window.location.origin}/auth/callback`,
      queryParams: {
        prompt:
          "select_account",
      },
    },
  });
```

Sistem juga menyediakan proses *login* menggunakan akun *Google*.

Potongan kode berikut digunakan untuk melakukan autentikasi *login* pengguna.

```
const { error } =
  await supabase.auth
    .signInWithOAuth({
      provider: "google",
      options: {
        redirectTo:
          `${window.location.origin}/auth/callback`,
      },
    });
```

Setelah proses *login* berhasil dilakukan, sistem akan mengambil data *role* pengguna dari *database* untuk menentukan hak akses pengguna di dalam sistem.

Potongan kode berikut digunakan untuk mengambil data *role* pengguna.

```
const {
  data: profile
} = await supabase
  .from("profiles")
  .select("role")
  .eq("id", userId)
  .single();
```

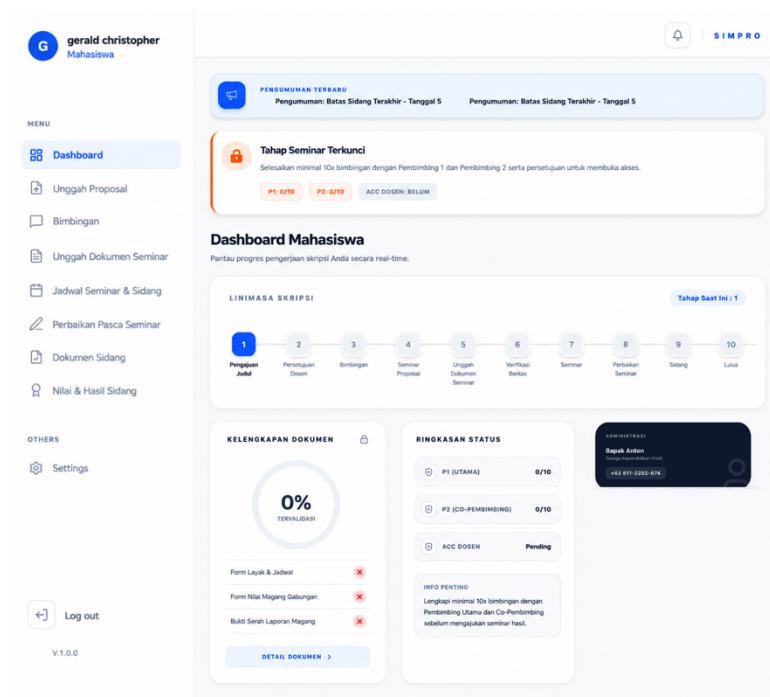
Sistem kemudian mengarahkan pengguna ke halaman *dashboard* sesuai dengan *role* masing-masing. Potongan kode berikut digunakan untuk melakukan proses *redirect* halaman berdasarkan *role* pengguna.

```
switch (
  profile.role
) {
  case "mahasiswa":
    router.push(
      "/mahasiswa/dashboard"
    );
    break;
  case "dosen":
    router.push(
      "/dosen/dashboarddosen"
    );
    break;

  case "kaprodi":
    router.push(
      "/kaprodi/dashboardkaprodi"
    );
    break;
  case "tendik":
    router.push(
      "/tendik/dashboardtendik"
    );
    break;
}
```

4.2.2 Halaman *Dashboard* Mahasiswa

Halaman *dashboard* mahasiswa merupakan halaman utama yang ditampilkan setelah mahasiswa berhasil melakukan *login* ke dalam sistem. Halaman ini dirancang untuk memberikan gambaran umum mengenai progres pengerjaan skripsi secara ringkas dan terstruktur, sehingga mahasiswa dapat memantau status akademiknya secara *real-time*.



Gambar 4.2 Halaman *dashboard* mahasiswa

Gambar 4.2 menampilkan tampilan *dashboard* mahasiswa setelah mahasiswa berhasil masuk ke sistem. Pada halaman ini terdapat linimasa skripsi (*timeline progress*) yang menunjukkan posisi tahapan akademik mahasiswa berdasarkan data yang tersimpan pada basis data. Sistem juga menampilkan status pemenuhan syarat mengajukan seminar hasil, progres kelengkapan dokumen, dan ringkasan jumlah bimbingan dengan dosen pembimbing utama maupun pendamping.

Kode *dashboard* mahasiswa mengimplementasikan mekanisme pengambilan data secara *asynchronous* menggunakan *library* SWR dan Supabase. Fungsi *fetcher* digunakan untuk mengambil data proposal, dokumen seminar, sesi bimbingan, data dosen pembimbing, serta pengumuman akademik yang aktif. Potongan kode berikut menunjukkan bahwa sistem akan memperbarui data

dashboard secara otomatis setiap 60 detik sehingga informasi progres skripsi dapat ditampilkan secara *real-time*.

```
const { data, error, isLoading } = useSWR(
  'dashboard_mahasiswa',
  fetcher,
  {
    revalidateOnFocus: true,
    refreshInterval: 60000
  }
);
```

Penentuan tahapan linimasa skripsi dilakukan berdasarkan status proposal, seminar, sidang, dan hasil verifikasi dokumen. Kode berikut digunakan untuk menentukan posisi mahasiswa pada linimasa skripsi secara dinamis berdasarkan proses akademik yang telah diselesaikan.

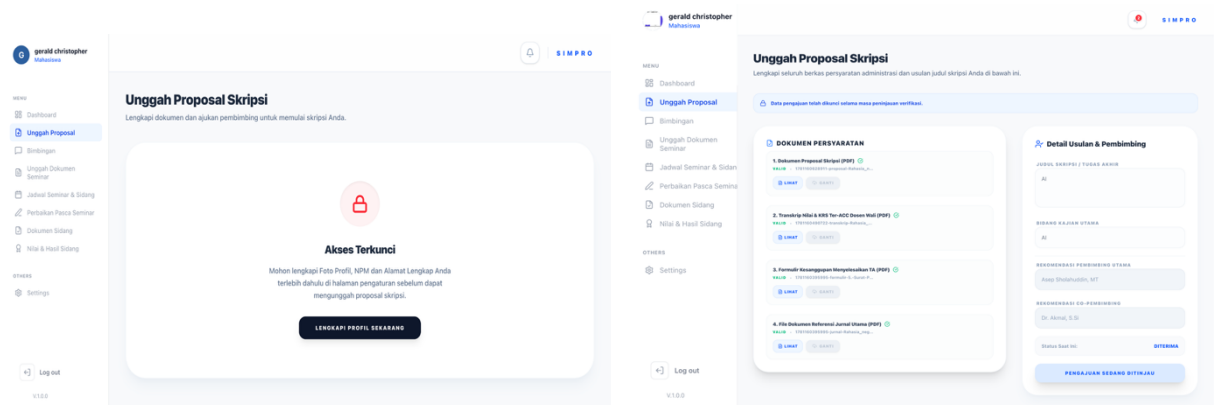
```
if (prop.status === "Diterima") {
  activeStep = 2;
  if (eligible) {
    activeStep = 3;
    if (uploadedDocsCount > 0)
      activeStep = 4;
    if (verifiedDocsCount > 0)
      activeStep = 5;
  }
}
```

Sistem juga melakukan validasi syarat seminar berdasarkan jumlah bimbingan dan persetujuan dosen pembimbing. Potongan kode berikut menunjukkan bahwa mahasiswa hanya dapat mengakses tahap seminar apabila telah memenuhi minimal 10 kali bimbingan pada masing-masing dosen pembimbing serta memperoleh persetujuan dari kedua dosen pembimbing.

```
const eligible =
  p1Count >= 10 &&
  p2Count >= 10 &&
  approvedByAll;
```


4.2.3 Halaman Unggah Proposal

Halaman unggah proposal skripsi menyediakan fasilitas bagi mahasiswa untuk mengunggah dokumen proposal skripsi sebagai tahap awal proses pengajuan skripsi. Halaman ini memungkinkan mahasiswa melengkapi dokumen serta mengajukan dosen pembimbing. Sistem menggunakan halaman ini sebagai pintu masuk proses *review* proposal skripsi.



(a)

(b)

Gambar 4.3 (a) Tampilan halaman unggah proposal sebelum mahasiswa melengkapi profil dan (b) tampilan halaman unggah proposal setelah mahasiswa telah melengkapi profil

Gambar 4.3 (a) menampilkan kondisi halaman ketika mahasiswa belum melengkapi data profil. Pada kondisi ini sistem akan mengunci akses pengajuan proposal sehingga mahasiswa tidak dapat melakukan unggah dokumen maupun pengajuan pembimbing.

Gambar 4.3 (b) menampilkan kondisi halaman setelah mahasiswa melengkapi profil. Pada kondisi ini sistem menyediakan formulir pengajuan

proposal yang terdiri atas unggah dokumen proposal skripsi, transkrip nilai dan KRS yang telah disetujui dosen wali, formulir kesanggupan menyelesaikan tugas akhir, serta dokumen referensi jurnal utama. Selain itu, mahasiswa juga dapat mengisi informasi judul skripsi, bidang kajian, serta memilih dosen pembimbing utama dan pembimbing pendamping sebagai rekomendasi pembimbing.

proposal yang terdiri atas unggah dokumen proposal skripsi, transkrip nilai dan KRS yang telah disetujui dosen wali, formulir kesanggupan menyelesaikan tugas akhir, serta dokumen referensi jurnal utama. Selain itu, mahasiswa juga dapat mengisi informasi judul skripsi, bidang kajian, serta memilih dosen pembimbing utama dan pembimbing pendamping sebagai rekomendasi pembimbing.

Sistem melakukan validasi kelengkapan profil mahasiswa sebelum membuka akses pengajuan proposal. Potongan kode berikut menunjukkan bahwa mahasiswa wajib melengkapi data NPM, foto profil, dan alamat terlebih dahulu.

```
const isProfileComplete = !!(
  profile?.npm &&
  profile?.avatar_url &&
  profile?.alamat
);
```

Setelah seluruh dokumen dipilih, sistem akan melakukan proses unggah berkas ke Supabase *Storage*. Setiap dokumen disimpan menggunakan nama file yang unik berdasarkan identitas pengguna dan timestamp sehingga dapat menghindari duplikasi nama file pada penyimpanan.

```
storagePath =
  `${user.id}/${Date.now()}-${fileToUpload.name}`;
await supabase.storage
  .from("proposals")
  .upload(storagePath, fileToUpload);
```

Proses yang sama diterapkan pada dokumen transkrip nilai, formulir pengajuan tugas akhir, dan dokumen referensi jurnal. Setelah seluruh berkas berhasil diunggah, sistem menyimpan data pengajuan proposal ke basis data dengan status awal "Menunggu Verifikasi Tendik" dan mengunci data selama proses peninjauan berlangsung.

```
await supabase
  .from("proposals")
  .insert({
    user_id: user.id,
    judul: judulSkripsi,
    bidang: bidangSkripsi,
    file_path: pathProposal,
    file_transkrip: pathTranskrip,
    file_formulir: pathFormulir,
    file_referensi: pathReferensi,
    status: "Menunggu Verifikasi Tendik",
    is_locked: true
  });
```

Selain menyimpan data proposal, sistem juga mencatat rekomendasi dosen pembimbing yang dipilih mahasiswa ke dalam tabel *proposal_recommendations*. Data yang disimpan meliputi identitas proposal, identitas dosen, dan tipe pembimbing yang dipilih, baik sebagai pembimbing utama maupun pembimbing pendamping.

```
const inserts = [
  {
    proposal_id: proposalId,
    dosen_id: pembimbing1,
    tipe: "pembimbing1"
  }
];
```

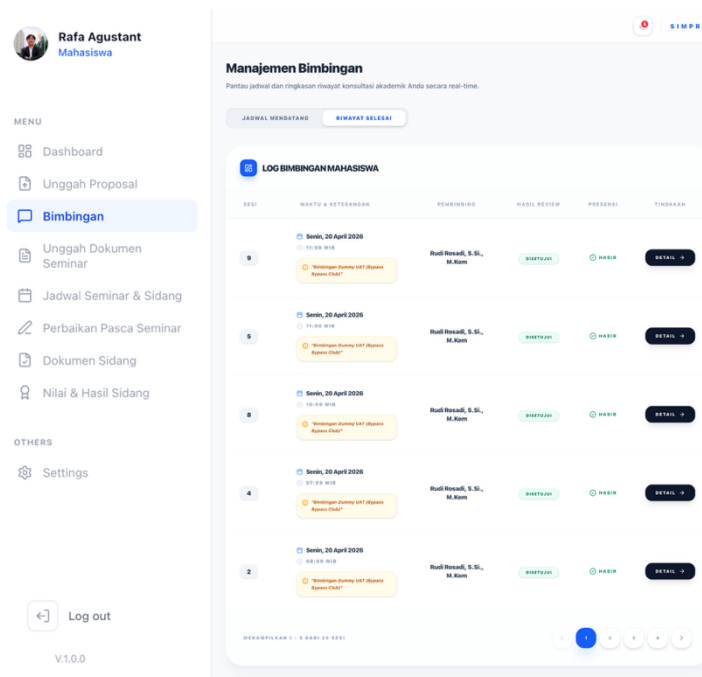
Sistem menyediakan mekanisme verifikasi bertahap terhadap dokumen yang diajukan. Pada tahap pertama, tendik melakukan pemeriksaan kelengkapan administrasi setiap dokumen. Apabila terdapat dokumen yang tidak memenuhi persyaratan, sistem akan mengubah status dokumen menjadi ditolak dan membuka

kembali akses unggah hanya pada dokumen yang memerlukan perbaikan. Sebaliknya, dokumen yang telah dinyatakan valid akan tetap terkunci sehingga tidak dapat diubah oleh mahasiswa.

Setelah seluruh dokumen administrasi dinyatakan lengkap, proposal akan diteruskan ke tahap persetujuan calon dosen pembimbing. Apabila proposal memerlukan revisi, sistem akan mengubah status menjadi Ditolak Dosen dan membuka kembali akses unggah dokumen proposal. Setelah mahasiswa mengunggah revisi proposal, sistem akan mengubah status menjadi menunggu persetujuan dosen dan mengunci kembali data selama proses peninjauan berlangsung.

4.2.4 Halaman Manajemen Bimbingan Mahasiswa

Halaman manajemen bimbingan mahasiswa digunakan oleh mahasiswa untuk memantau jadwal dan riwayat proses bimbingan skripsi secara *real-time* pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Melalui halaman ini, mahasiswa dapat melihat jadwal konsultasi dengan dosen pembimbing, status kehadiran, hasil revisi, serta detail sesi bimbingan yang telah dilakukan.



Gambar 4.4 Halaman manajemen bimbingan mahasiswa

Gambar 4.4 menampilkan tampilan halaman manajemen bimbingan mahasiswa. Pada halaman ini mahasiswa dapat mengakses dua *tab* utama, yaitu jadwal mendatang dan riwayat selesai. *Tab* jadwal mendatang digunakan untuk melihat sesi bimbingan yang akan berlangsung, sedangkan tab riwayat selesai digunakan untuk melihat hasil konsultasi yang telah selesai beserta status revisi dari dosen pembimbing.

Sistem mengambil data bimbingan mahasiswa secara *asynchronous* menggunakan *library* SWR dan Supabase sehingga informasi jadwal dapat diperbarui secara otomatis. Potongan kode berikut digunakan agar mahasiswa dapat memperoleh informasi bimbingan terbaru ketika kembali membuka halaman tanpa perlu memuat ulang sistem secara manual.

```
const { data: rows = [], isLoading } =
  useSWR(
    'list_bimbingan_mhs',
    fetchBimbinganData,
    {
      revalidateOnFocus: true,
    }
  );
```

Sebelum data bimbingan ditampilkan, sistem akan memeriksa status proposal mahasiswa. Data bimbingan hanya dapat diakses apabila proposal skripsi telah diterima oleh dosen pembimbing. Kode berikut digunakan untuk memastikan bahwa fitur bimbingan hanya tersedia bagi mahasiswa yang telah lolos tahap pengajuan proposal dan telah ditetapkan pembimbingnya oleh kaprodi.

```
if (proposalData?.status !== "Diterima") {
  return [];
}
```

Halaman manajemen bimbingan juga memisahkan data menjadi kategori jadwal aktif dan riwayat selesai berdasarkan status sesi bimbingan. Potongan kode berikut digunakan agar mahasiswa dapat lebih mudah membedakan sesi bimbingan yang masih berlangsung dengan sesi yang telah selesai dan mendapatkan hasil revisi.

```
const filteredRows =
  activeTab === "jadwal"
    ? rows.filter(r =>
      r.status !== "selesai" &&
      r.status !== "revisi" &&
      r.status !== "dibatalkan"
    )
    : rows.filter(r =>
      (r.feedback === "disetujui" ||
      r.feedback === "revisi") &&
      r.kehadiran === "hadir"
    );
```

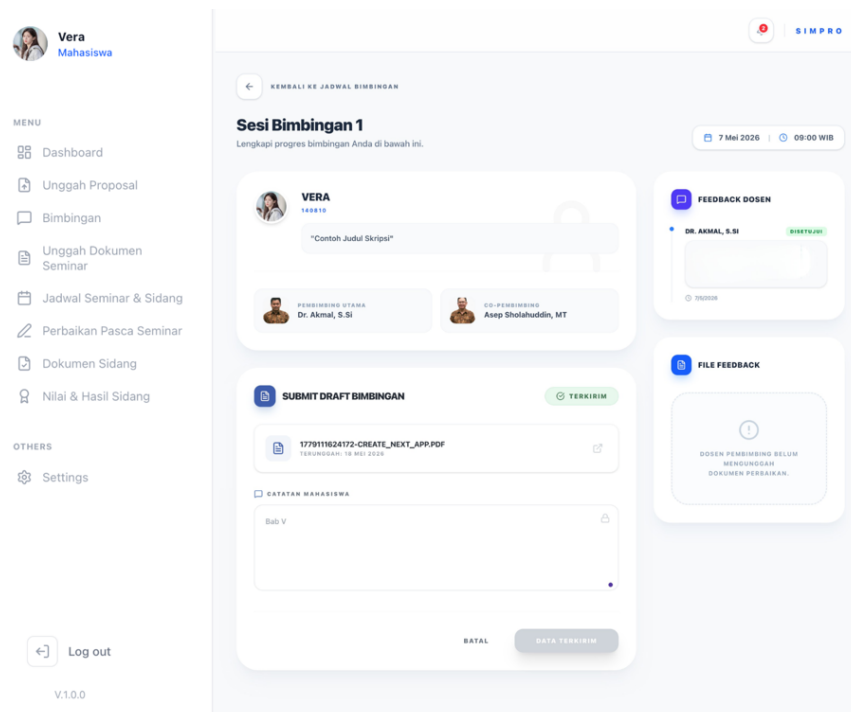
Sistem juga menerapkan fitur *pagination* untuk membatasi jumlah data bimbingan yang ditampilkan dalam satu halaman. Kode berikut digunakan untuk

membagi data bimbingan ke dalam beberapa halaman sehingga tampilan tabel tetap rapi dan mudah dibaca oleh mahasiswa.

```
const currentData =
  filteredRows.slice(
    indexOfFirstItem,
    indexOfLastItem
  );
```

4.2.5 Halaman Detail Sesi Bimbingan Mahasiswa

Halaman detail sesi bimbingan digunakan oleh mahasiswa untuk melihat detail sesi bimbingan skripsi serta mengunggah draf hasil pengerjaan kepada dosen pembimbing. Pada halaman ini mahasiswa dapat melihat informasi jadwal bimbingan, dosen pembimbing, mengunggah draft bimbingan, menambahkan catatan perkembangan, serta melihat *feedback* dari dosen pembimbing.



Gambar 4.5 Halaman detail sesi bimbingan mahasiswa

Gambar 4.5 menampilkan tampilan halaman detail sesi bimbingan mahasiswa. Sistem menampilkan informasi sesi bimbingan beserta fitur unggah draf dan *feedback* dosen pembimbing.

Sistem mengambil data sesi bimbingan berdasarkan id sesi yang dipilih. Potongan kode berikut digunakan untuk memperoleh data sesi bimbingan beserta data proposal dan mahasiswa.

```
const {
  data: sessionData
} = await supabase
  .from(
    "guidance_sessions"
  )
  .select(`
    id,
    sesi_ke,
    tanggal,
    jam,
    status,
    dosen_id,
    proposal:proposals (
      id,
      judul,
      user:profiles (
        nama,
        npm,
        avatar_url
      )
    )
  `)
  .eq("id", sessionId)
  .single();
```

Sistem mengambil data dosen pembimbing berdasarkan proposal skripsi mahasiswa. Potongan kode berikut digunakan untuk memperoleh data dosen pembimbing utama dan pendamping.


```
const {
  data: supervisors
} = await supabase
  .from(
    "thesis_supervisors"
  )
  .select(`
    role,
    dosen_id,
    dosen:profiles (
      nama,
      avatar_url
    )
  `)
  .eq(
    "proposal_id",
    proposalId
  );
```

Sistem mengambil data draf bimbingan yang telah diunggah mahasiswa.

Potongan kode berikut digunakan untuk memperoleh data file draf bimbingan mahasiswa.

```
const {
  data: sessionDrafts
} = await supabase
  .from(
    "session_drafts"
  )
  .select(`
    id,
    file_url,
    uploaded_at,
    mahasiswa_id,
    catatan
  `)
  .eq(
    "session_id",
    sessionId
  )
  .order(
    "uploaded_at",
    { ascending: false }
  );
```

Sistem mengambil data *feedback* dosen pembimbing terhadap draf bimbingan mahasiswa. Potongan kode berikut digunakan untuk memperoleh komentar dan file *feedback* dosen pembimbing.

```

const {
  data: feedbackData
} = await supabase
  .from(
    "session_feedbacks"
  )
  .select(`
    id,
    komentar,
    file_url,
    status_revisi,
    created_at,
    dosen:profiles (
      nama
    )
  `)
  .eq(
    "session_id",
    sessionId
  )
  .order(
    "created_at",
    { ascending: false }
  );

```

Sistem memungkinkan mahasiswa mengunggah draf bimbingan ke penyimpanan Supabase *Storage*. Potongan kode berikut digunakan untuk mengunggah file draf bimbingan mahasiswa.

```

const {
  error: uploadError
} = await supabase
  .storage
  .from("draftsession")
  .upload(
    filePath,
    file
  );

```

Sistem menyimpan data draf bimbingan mahasiswa ke *database*. Potongan kode berikut digunakan untuk menyimpan informasi draf dan catatan mahasiswa.

```

await supabase
  .from(
    "session_drafts"
  )
  .insert({
    session_id:
      sessionId,
    mahasiswa_id:
      userId,
    file_url:
      urlData.publicUrl,
    catatan:
      catatanAktif,
  });

```

Sistem menyediakan fitur untuk membuka draf bimbingan dan file *feedback* dosen menggunakan *signed* URL. Potongan kode berikut digunakan untuk menghasilkan URL sementara file bimbingan mahasiswa.

```

const { data } =
  await supabase.storage
    .from("draftsession")
    .createSignedUrl(
      path,
      3600
    );

```

Sistem mengirimkan notifikasi otomatis kepada dosen pembimbing setelah mahasiswa mengunggah draf bimbingan. Potongan kode berikut digunakan untuk mengirimkan notifikasi kepada dosen pembimbing.

```

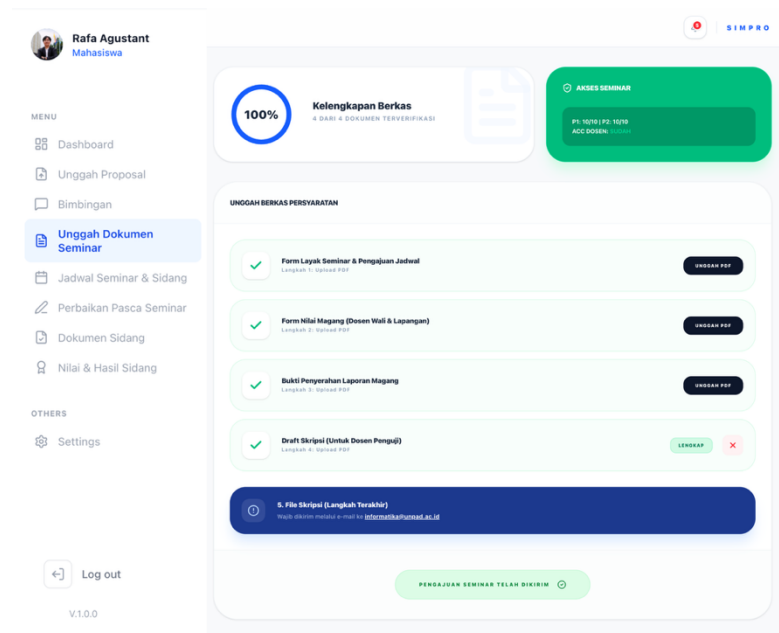
await sendNotification(
  targetDosenId,
  "Draft Bimbingan Baru",
  `${data?.proposal?.user?.nama}
  telah mengunggah draft
  untuk sesi bimbingan.`
);

```

4.2.6 Halaman Unggah Dokumen Seminar

Halaman unggah dokumen seminar digunakan oleh mahasiswa untuk melengkapi persyaratan administrasi seminar skripsi pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Pada halaman ini mahasiswa dapat

mengunggah dokumen persyaratan seminar, memantau progres kelengkapan berkas, serta melihat status validasi syarat seminar berdasarkan jumlah bimbingan dan persetujuan dosen pembimbing.



Gambar 4.6 Halaman unggah dokumen seminar

Gambar 4.6 menampilkan tampilan halaman unggah dokumen seminar. Halaman ini menampilkan persentase kelengkapan dokumen seminar, status akses seminar, serta daftar dokumen yang wajib diunggah mahasiswa. Sistem juga menyediakan informasi status verifikasi setiap dokumen, seperti menunggu verifikasi atau lengkap.

Sistem melakukan validasi syarat seminar berdasarkan jumlah bimbingan dan persetujuan dosen pembimbing. Potongan kode berikut menunjukkan bahwa mahasiswa hanya dapat mengakses proses unggah dokumen seminar apabila telah memenuhi minimal 10 kali bimbingan pada masing-masing dosen pembimbing serta memperoleh persetujuan dari kedua dosen pembimbing.

```
const isEligible =
  p1 >= 10 &&
  p2 >= 10 &&
  approvedByAll;
```

Halaman unggah dokumen seminar juga menampilkan persentase kelengkapan dokumen yang telah diverifikasi oleh sistem. Kode berikut digunakan untuk menghitung jumlah dokumen yang telah dinyatakan lengkap sehingga mahasiswa dapat memantau progres persyaratan seminar secara real-time.

```
const verifiedCount =
  seminarDocs.filter(
    d => documents[d.id]?.status === 'Lengkap'
  ).length;
const percentage =
  Math.round(
    (verifiedCount / totalDocs) * 100
  );
```

Proses unggah dokumen dilakukan menggunakan Supabase *Storage* melalui fungsi *handleUpload*. Potongan kode berikut digunakan untuk menyimpan dokumen seminar ke *cloud storage* dengan nama file yang unik berdasarkan identitas pengguna dan waktu unggah.

```
const filePath =
  `${userId}/${docId}_${Date.now()}.pdf`;
await supabase.storage
  .from('docseminar')
  .upload(filePath, file, {
    upsert: true
  });
```

Setelah dokumen berhasil diunggah, sistem akan menyimpan metadata dokumen ke tabel *seminar_documents* beserta status verifikasi.

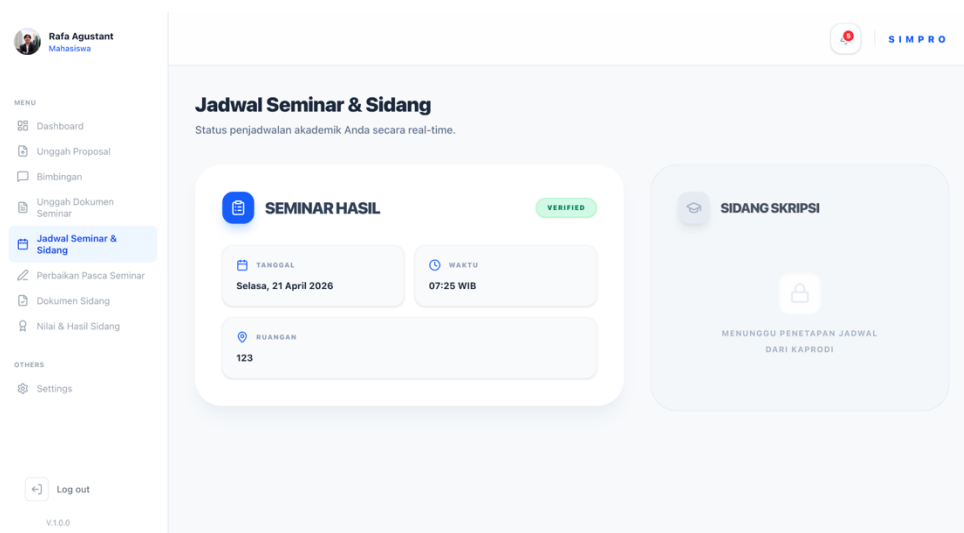
```
await supabase
  .from('seminar_documents')
  .upsert({
    proposal_id: proposalId,
    nama_dokumen: docId,
    file_url: filePath,
    status: statusAwal});
```

Mahasiswa juga dapat mengirim pengajuan seminar setelah seluruh dokumen dinyatakan lengkap oleh sistem. Kode berikut digunakan untuk mencatat pengajuan seminar mahasiswa ke basis data dengan status awal menunggu persetujuan sehingga dapat diproses lebih lanjut oleh kaprodi.

```
await supabase
  .from('seminar_requests')
  .upsert({
    proposal_id: proposalId,
    tipe: 'seminar',
    status: 'Menunggu Persetujuan'
  });
```

4.2.7 Halaman Jadwal Seminar dan Sidang

Halaman jadwal seminar dan sidang digunakan oleh mahasiswa untuk melihat informasi penjadwalan seminar hasil dan sidang skripsi pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Melalui halaman ini mahasiswa dapat mengetahui tanggal pelaksanaan, waktu, ruangan dan jadwal yang telah ditetapkan oleh kaprodi.



Gambar 4.7 Halaman jadwal seminar dan sidang

Gambar 4.7 menampilkan tampilan halaman jadwal seminar dan sidang.

Pada halaman ini sistem membagi informasi menjadi dua bagian utama, yaitu seminar hasil dan sidang skripsi. Bagian seminar menampilkan informasi jadwal seminar yang telah ditetapkan, sedangkan bagian sidang akan menampilkan status penjadwalan sidang skripsi apabila jadwal telah ditentukan oleh kaprodi.

Sistem mengambil data jadwal seminar dan sidang mahasiswa secara *asynchronous* menggunakan SWR dan Supabase. Potongan kode berikut digunakan agar mahasiswa dapat memperoleh pembaruan jadwal seminar dan sidang secara otomatis tanpa perlu memuat ulang halaman secara manual.

```
const { data, error, isLoading } =
  useSWR(
    'jadwal_mahasiswa_all',
    fetchSchedules,
    {
      revalidateOnFocus: true,
      refreshInterval: 60000
    }
  );
```

Sistem juga memeriksa apakah mahasiswa telah memiliki jadwal seminar maupun sidang yang valid. Kode berikut digunakan untuk menentukan apakah informasi jadwal seminar dan sidang dapat ditampilkan kepada mahasiswa atau masih berada dalam status menunggu penjadwalan.

```
const hasSeminarSchedule =
  !!semSchedule?.tanggal;
const isSidangScheduled =
  !!sidangData?.tanggal_sidang;
```

Informasi jadwal seminar yang telah tersedia kemudian ditampilkan berdasarkan data tanggal, waktu, dan ruangan seminar. Potongan kode berikut digunakan untuk menampilkan detail pelaksanaan seminar hasil yang telah ditetapkan oleh program studi.

```

<ScheduleInfo
  label="Tanggal"
  value={formatDate(semSchedule?.tanggal)}
/>
<ScheduleInfo
  label="Waktu"
  value={`$${semSchedule?.jam?.slice(0,5)} WIB`}
/>

```

Apabila jadwal sidang belum ditentukan, sistem akan menampilkan status menunggu penetapan jadwal dari kaprodi. Kode berikut digunakan untuk memberikan informasi kepada mahasiswa bahwa jadwal sidang skripsi masih dalam proses penjadwalan oleh pihak program studi.

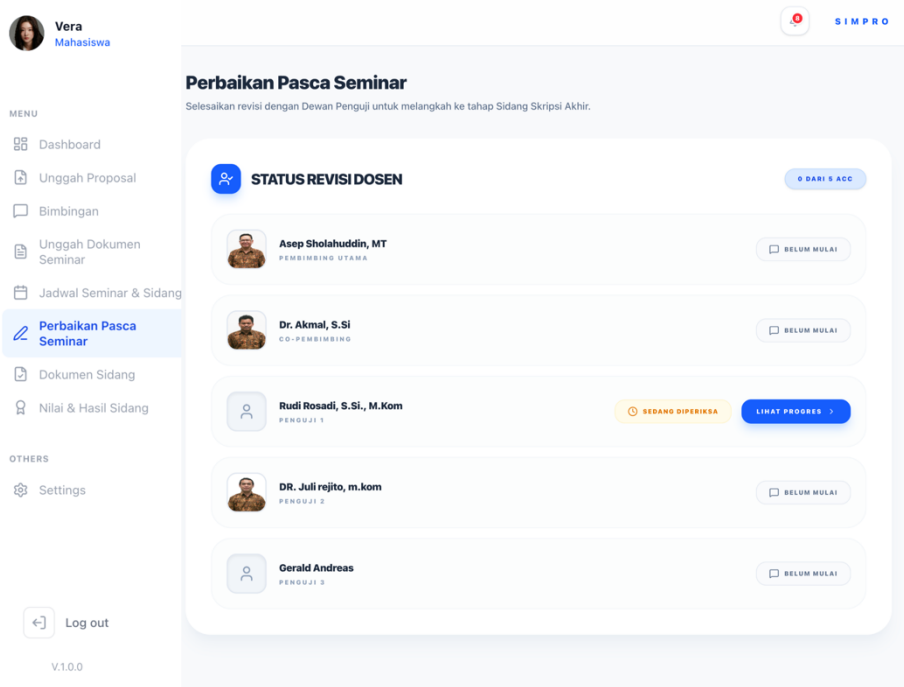
```

{!isSidangScheduled && (
  <p>
    Menunggu Penetapan Jadwal
    dari Kaprodi
  </p>
)}

```

4.2.8 Halaman Perbaikan Pasca Seminar Mahasiswa

Halaman perbaikan pasca seminar digunakan oleh mahasiswa untuk memantau dan menyelesaikan revisi hasil seminar skripsi berdasarkan masukan dari dosen pembimbing dan dosen penguji. Melalui halaman ini mahasiswa dapat melihat status revisi masing-masing dosen, progres persetujuan revisi, serta mengakses detail catatan revisi yang harus diperbaiki sebelum melanjutkan ke tahap sidang akhir.



Gambar 4.8 Halaman perbaikan pasca seminar mahasiswa

Gambar 4.8 menampilkan tampilan halaman perbaikan pasca seminar. Pada halaman ini sistem menampilkan daftar dosen pembimbing dan penguji beserta status revisi masing-masing, seperti belum mulai, sedang diperiksa, perlu diperbaiki, dan revisi diterima. Mahasiswa juga dapat melihat jumlah dosen yang telah memberikan *ACC* revisi secara *real-time*.

Sistem mengambil data revisi seminar mahasiswa secara *asynchronous* menggunakan SWR dan Supabase. Potongan kode berikut digunakan agar mahasiswa dapat memperoleh pembaruan status revisi dosen secara otomatis tanpa perlu memuat ulang halaman secara manual.

```
const { data, error, isLoading } =
  useSWR(
    'revisi_mahasiswa_data_full',
    fetchRevisiData,
    {
      revalidateOnFocus: true,
      refreshInterval: 30000
    }
  );
```

Sistem juga memeriksa apakah seluruh dosen pembimbing dan penguji telah memberikan *ACC* revisi. Kode berikut digunakan untuk menentukan apakah mahasiswa telah menyelesaikan seluruh revisi seminar sehingga dapat melanjutkan ke tahap berikutnya.

```
isAllAcc =
  mappedExaminers.length > 0 &&
  mappedExaminers.every(
    e => e.status_revisi === 'diterima'
  );
```

Halaman perbaikan pasca seminar menampilkan jumlah dosen yang telah memberikan persetujuan revisi. Potongan kode berikut digunakan untuk menghitung total dosen yang telah memberikan *ACC* revisi sehingga mahasiswa dapat memantau progres penyelesaian revisi seminar.

```
examiners.filter(
  e => e.status_revisi === 'diterima'
).length
```

Sistem juga menyediakan akses menuju detail revisi masing-masing dosen berdasarkan status revisi yang diberikan. Kode berikut digunakan untuk mengarahkan mahasiswa menuju halaman detail revisi sehingga mahasiswa dapat melihat catatan revisi dan memberikan jawaban terhadap masukan dosen pembimbing maupun penguji.

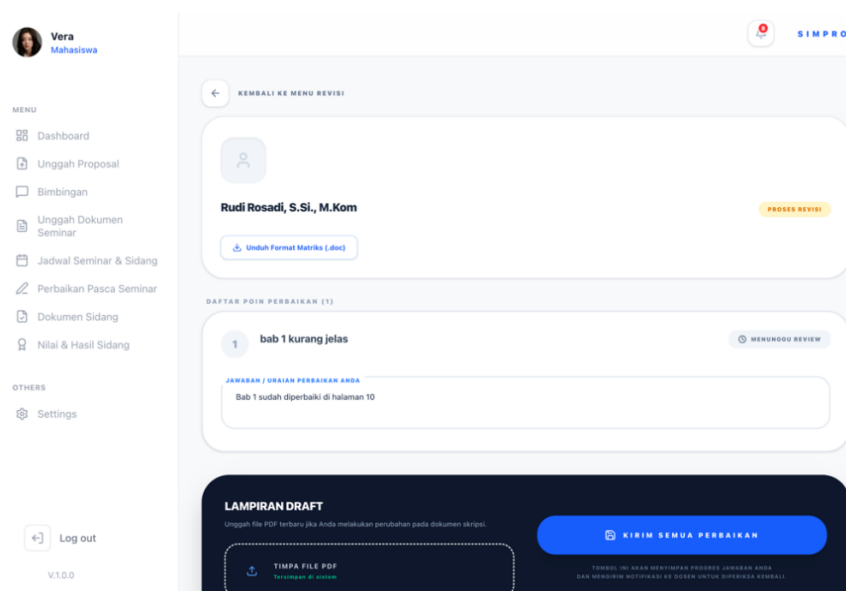
```
router.push(
  `/mahasiswa/revisiseminar?dosen_id=${penguji.dosen_id}`
)
```

Apabila seluruh revisi telah disetujui, mahasiswa dapat mengunduh dokumen matriks perbaikan final yang telah berisi catatan revisi, jawaban mahasiswa, dan tanda tangan dosen pembimbing maupun penguji. Potongan kode berikut digunakan untuk menghasilkan dokumen matriks perbaikan final sebagai syarat administrasi sebelum mahasiswa melanjutkan ke tahap sidang skripsi akhir.

```
if (allAcc) {
    handleExportFullMatriks();
}
```

4.2.9 Halaman Detail Perbaikan Pasca Seminar Mahasiswa

Halaman detail perbaikan pasca seminar digunakan oleh mahasiswa untuk melihat detail catatan revisi dari dosen pembimbing maupun dosen penguji setelah pelaksanaan seminar. Melalui halaman ini mahasiswa dapat membaca poin revisi, menuliskan jawaban atau uraian perbaikan, mengunggah draf skripsi terbaru, serta mengirim revisi kembali kepada dosen untuk diperiksa.



Gambar 4.9 Halaman detail perbaikan pasca seminar mahasiswa

Gambar 4.9 menampilkan tampilan halaman detail perbaikan pasca seminar mahasiswa. Pada halaman ini sistem menampilkan identitas dosen, status revisi, daftar poin perbaikan, area jawaban revisi mahasiswa, serta fitur unggah draft skripsi terbaru. Sistem juga menyediakan fitur unduh format matriks perbaikan dalam bentuk dokumen .doc.

Sistem mengambil data revisi dosen secara *asynchronous* menggunakan SWR dan Supabase berdasarkan dosen_id yang dipilih mahasiswa. Potongan kode berikut digunakan untuk menampilkan detail revisi dari dosen tertentu secara dinamis sesuai pilihan mahasiswa.

```
const { data, error, isLoading } =
  useSWR(
    isMounted && dosen_id
      ? `revisi_mhs_v5_${dosen_id}`
      : null,
    () => fetchRuangRevisi(dosen_id)
  );
```

Sistem melakukan parsing data revisi agar daftar poin perbaikan dapat ditampilkan dalam bentuk terstruktur. Kode berikut digunakan untuk mengubah data revisi berbentuk JSON menjadi daftar poin revisi yang dapat ditampilkan dan diedit oleh mahasiswa.

```
const parsedCatatan =
  parseSafe(feedback.catatan_revisi);

const parsedJawaban =
  parseSafe(feedback.jawaban_revisi);
```

Mahasiswa dapat menuliskan jawaban atau uraian perbaikan pada setiap poin revisi yang diberikan dosen. Potongan kode berikut digunakan untuk menyimpan perubahan jawaban revisi mahasiswa secara sementara sebelum dikirim ke sistem.

```
updated[idx].jawaban_mhs = text;
setPointAnswers(updated);
```

Sistem juga menyediakan fitur unggah draf skripsi terbaru apabila mahasiswa melakukan perubahan pada dokumen skripsi. Kode berikut digunakan untuk menyimpan file draf revisi mahasiswa ke *cloud storage* agar dapat diperiksa kembali oleh dosen pembimbing maupun penguji.

```
await supabase.storage
  .from('seminar_perbaikan')
  .upload(filePath, fileUploadForm, {
    upsert: true
  });
```

Setelah seluruh jawaban revisi selesai diisi, mahasiswa dapat mengirim revisi kembali kepada dosen untuk diperiksa ulang. Potongan kode berikut digunakan untuk menyimpan jawaban revisi mahasiswa ke basis data serta mengubah status revisi menjadi *diperiksa* sehingga dosen dapat melakukan proses review kembali.

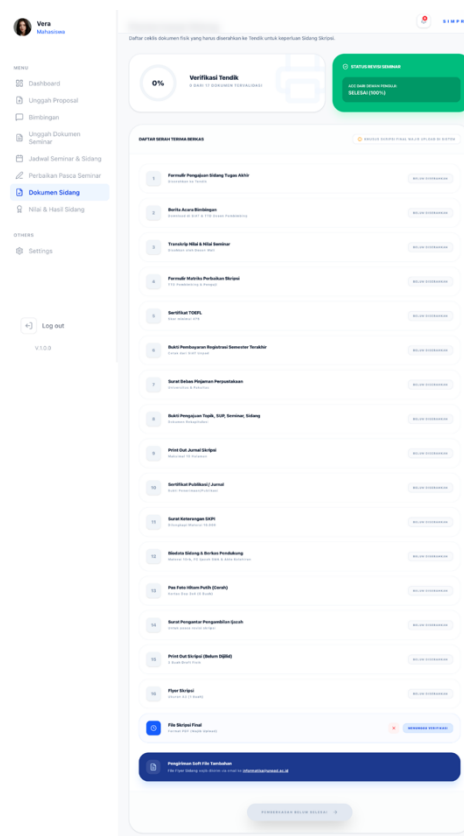
```
await supabase
  .from('seminar_feedbacks')
  .update({
    jawaban_revisi:
      JSON.stringify(
        pointAnswers.map(pa => ({
          answerText:
            pa.jawaban_mhs
        })))
  },
  {
    status_revisi: 'diperiksa'
  })
```

Sistem juga menyediakan fitur ekspor dokumen matriks perbaikan dalam format .doc. Kode berikut digunakan untuk menghasilkan dokumen matriks perbaikan yang berisi catatan revisi, jawaban mahasiswa, serta tanda tangan dosen sebagai dokumen administrasi revisi seminar skripsi.

```
const blob = new Blob(
  ['\\u0000', html],
  { type: 'application/msword' }
);
```

4.2.10 Halaman Unggah Dokumen Sidang

Halaman unggah dokumen sidang digunakan oleh mahasiswa untuk melengkapi persyaratan administrasi sidang skripsi pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Melalui halaman ini mahasiswa dapat mengunggah berbagai dokumen persyaratan sidang, memantau status verifikasi berkas oleh tendik, serta melihat progres kelengkapan dokumen secara *real-time*.



Gambar 4.10 Halaman unggah dokumen sidang

Gambar 4.10 menampilkan tampilan halaman unggah dokumen sidang.

Pada halaman ini sistem menampilkan daftar dokumen administrasi sidang, status

verifikasi masing-masing dokumen, persentase kelengkapan berkas, serta status *ACC* revisi seminar dari dosen pembimbing dan penguji. Sistem juga menyediakan fitur unggah file skripsi *final* dalam format PDF.

Sistem melakukan validasi bahwa mahasiswa hanya dapat mengakses proses pemberkasan sidang apabila seluruh revisi seminar telah memperoleh persetujuan dosen pembimbing dan penguji. Potongan kode berikut menunjukkan bahwa mahasiswa hanya dapat melanjutkan proses unggah dokumen sidang apabila seluruh dosen yang terlibat telah memberikan *ACC* revisi seminar.

```
allAcc =
  accCount === allDosenIds.size;
```

Halaman unggah dokumen sidang juga menampilkan progres verifikasi dokumen berdasarkan jumlah dokumen yang telah divalidasi oleh tendik. Kode berikut digunakan untuk menghitung persentase kelengkapan dokumen sidang secara *real-time* sehingga mahasiswa dapat memantau progres pemberkasan.

```
const verifiedCount =
  sidangDocsList.filter(
    d => documents[d.id]?.status === 'Lengkap'
  ).length;

const percentage =
  Math.round(
    (verifiedCount / totalDocs) * 100
  );
```

Proses unggah file skripsi *final* dilakukan menggunakan Supabase *Storage*. Potongan kode berikut digunakan untuk menyimpan file skripsi *final* mahasiswa ke *cloud storage* sebelum dilakukan proses verifikasi oleh tendik.

```
await supabase.storage
  .from('docseminar')
  .upload(filePath, file);
```

Setelah file berhasil diunggah, sistem akan menyimpan metadata dokumen ke tabel *sidang_documents_verification*. Kode berikut digunakan untuk mencatat dokumen yang telah diunggah mahasiswa beserta status verifikasi pada basis data.

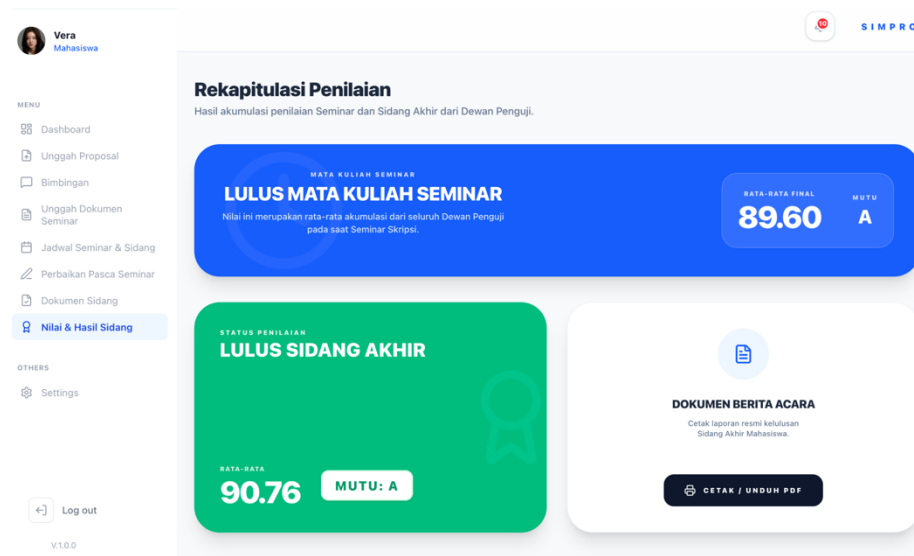
```
await supabase
  .from('sidang_documents_verification')
  .upsert({
    proposal_id: proposalId,
    nama_dokumen: 'file_skripsi_final',
    status: 'Menunggu Verifikasi',
    file_url: filePath
  });
```

Mahasiswa juga dapat mengajukan penjadwalan sidang setelah seluruh dokumen berhasil diverifikasi oleh tendik. Potongan kode berikut digunakan untuk mengirim pengajuan sidang skripsi mahasiswa ke pihak program studi agar jadwal sidang dapat ditetapkan oleh kaprodi.

```
await supabase
  .from("sidang_requests")
  .insert({
    proposal_id: proposalId,
    seminar_request_id: seminarId,
    status: "menunggu_penjadwalan",
  });
```

4.2.11 Halaman Nilai dan Hasil Sidang

Halaman nilai dan hasil sidang digunakan oleh mahasiswa untuk melihat rekapitulasi hasil penilaian seminar dan sidang skripsi akhir pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Melalui halaman ini mahasiswa dapat mengetahui nilai rata-rata seminar, nilai sidang akhir, huruf mutu, status kelulusan, serta mencetak dokumen berita acara sidang secara langsung dari sistem.



Gambar 4.11 Halaman nilai dan hasil sidang

Gambar 4.11 menampilkan tampilan halaman nilai dan hasil sidang. Pada halaman ini sistem menampilkan hasil akumulasi nilai seminar dan sidang akhir dari dosen pembimbing maupun dosen penguji. Selain itu, sistem juga menyediakan fitur cetak berita acara sidang dalam format PDF yang dapat digunakan sebagai dokumen administrasi akademik mahasiswa.

Sistem mengambil data nilai seminar dan sidang mahasiswa secara *asynchronous* menggunakan Supabase. Potongan kode berikut digunakan untuk mengambil data nilai sidang dari seluruh dosen penguji berdasarkan sidang yang dimiliki mahasiswa.

```
const { data: sidGrades } =
  await supabase
    .from('sidang_grades')
    .select('dosen_id, nilai_akhir')
    .eq(
      'sidang_request_id',
      sidangReq?.id
    );
```

Sistem kemudian menghitung nilai rata-rata seminar berdasarkan akumulasi nilai dari dosen pembimbing dan penguji seminar. Kode berikut digunakan untuk menghasilkan nilai akhir seminar mahasiswa yang kemudian ditampilkan pada halaman rekapitulasi nilai.

```
const rataSem =
    totalNilaiSem / 5;

setNilaiSeminar(rataSem);
```

Selain nilai seminar, sistem juga menghitung nilai rata-rata sidang akhir beserta huruf mutu mahasiswa. Potongan kode berikut digunakan untuk menentukan hasil akhir sidang mahasiswa berdasarkan akumulasi penilaian seluruh dewan penguji.

```
const rataRata =
    totalNilaiSidang / 5;
setNilaiAkhir(rataRata);
if (rataRata >= 80)
    setHurufMutu("A");
```

Sistem juga memeriksa apakah seluruh dosen telah mengisi nilai sidang sebelum status kelulusan ditampilkan kepada mahasiswa. Kode berikut digunakan untuk memastikan bahwa hasil sidang hanya dapat ditampilkan apabila seluruh dosen pembimbing dan penguji telah menyelesaikan proses penilaian.

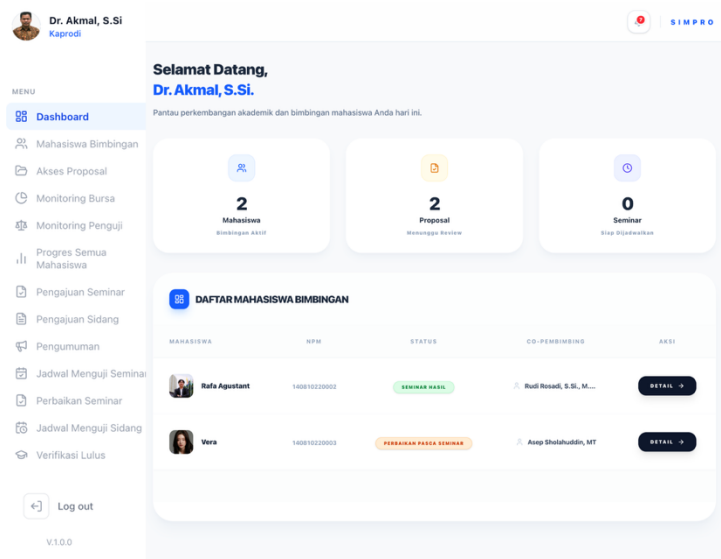
```
if (countNilaiSidang === 5) {
    setIsComplete(true);
}
```

Mahasiswa juga dapat mencetak dokumen berita acara sidang melalui fitur cetak PDF yang tersedia pada halaman sistem. Potongan kode berikut digunakan untuk menghasilkan dokumen berita acara sidang dalam format cetak yang dapat diunduh maupun dicetak langsung oleh mahasiswa.

```
printWindow.print();
```

4.2.12 Halaman *Dashboard* Ketua Program Studi

Halaman *dashboard* ketua program studi digunakan oleh kaprodi untuk memantau perkembangan akademik mahasiswa bimbingan pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Melalui halaman ini kaprodi dapat melihat jumlah mahasiswa bimbingan aktif, proposal yang menunggu *review*, seminar yang siap dijadwalkan, serta daftar mahasiswa beserta tahapan skripsinya secara *real-time*.



Gambar 4.12 Halaman *dashboard* ketua program studi

Gambar 4.12 menampilkan tampilan halaman *dashboard* kaprodi. Pada halaman ini sistem menampilkan ringkasan statistik akademik dalam bentuk kartu informasi (*stat card*) serta tabel daftar mahasiswa bimbingan yang memuat nama mahasiswa, NPM, status tahapan skripsi, *co*-pembimbing, dan tombol detail monitoring mahasiswa.

Sistem mengambil data *dashboard* kaprodi secara *asynchronous* menggunakan SWR dan Supabase. Potongan kode berikut digunakan agar data

dashboard dapat diperbarui secara otomatis setiap satu menit sehingga kaprodi dapat memantau perkembangan akademik mahasiswa secara *real-time*.

```
const { data: cache, isLoading } =
  useSWR(
    'dashboard_kaprodi_main',
    fetchDashboardKaprodiData,
    {
      revalidateOnFocus: true,
      refreshInterval: 60000
    }
  );
```

Sistem menghitung jumlah seminar yang siap dijadwalkan berdasarkan data seminar yang belum memiliki jadwal seminar. Kode berikut digunakan untuk menampilkan jumlah mahasiswa yang telah memenuhi syarat seminar namun belum memperoleh jadwal seminar dari program studi.

```
const totalSeminarSiap =
  (seminarData || []).filter(
    (s: any) =>
      !s.seminar_schedules ||
      s.seminar_schedules.length === 0
  ).length;
```

Sistem menghitung jumlah proposal mahasiswa yang masih menunggu penentuan dosen pembimbing. Potongan kode berikut digunakan untuk membantu kaprodi memantau proposal mahasiswa yang masih membutuhkan proses *review* dan penetapan dosen pembimbing.

```
const totalProposalPending =
  (proposalData || []).filter(
    (p: any) =>
      !p.thesis_supervisors ||
      p.thesis_supervisors.length === 0
  ).length;
```

Sistem juga melakukan validasi kelayakan seminar mahasiswa berdasarkan jumlah bimbingan dan persetujuan dosen pembimbing. Kode berikut digunakan

untuk menentukan apakah mahasiswa telah memenuhi syarat mengikuti seminar hasil berdasarkan jumlah bimbingan dan persetujuan dosen pembimbing.

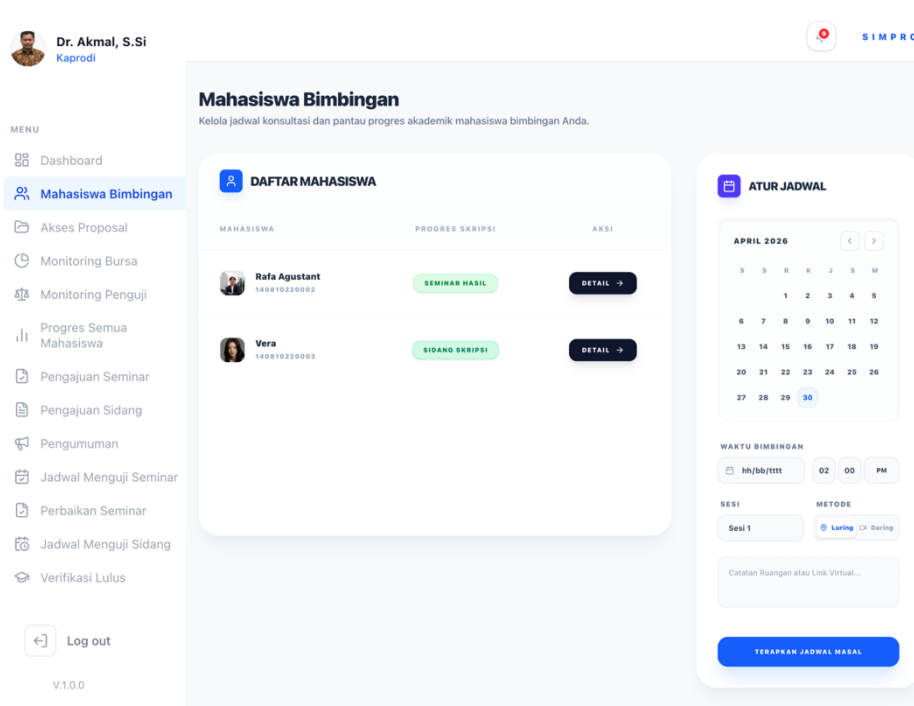
```
const isEligible =
  p1Count >= 10 &&
  p2Count >= 10 &&
  approvedByAll;
```

Tahapan skripsi mahasiswa kemudian ditampilkan secara dinamis berdasarkan status proposal, seminar, revisi, maupun sidang mahasiswa. Potongan kode berikut digunakan untuk memperbarui status tahapan skripsi mahasiswa sehingga kaprodi dapat memantau perkembangan akademik setiap mahasiswa secara terstruktur melalui dashboard sistem.

```
if (hasSidang) {
  finalTahap =
    "Sidang Skripsi";
}
```

4.2.13 Halaman Bimbingan Mahasiswa (Kaprodi & Dosen)

Halaman bimbingan mahasiswa digunakan oleh ketua program studi dan dosen untuk memantau progres skripsi mahasiswa bimbingan sekaligus mengatur jadwal konsultasi akademik secara massal pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Melalui halaman ini kaprodi dan dosen dapat melihat daftar mahasiswa aktif, status progres skripsi, serta membuat jadwal bimbingan berdasarkan tanggal, sesi, metode bimbingan, dan waktu pelaksanaan.



Gambar 4.13 Halaman bimbingan mahasiswa

Gambar 4.13 menampilkan tampilan halaman bimbingan mahasiswa untuk kaprodi dan dosen. Pada halaman ini sistem membagi antarmuka menjadi dua bagian utama, yaitu tabel daftar mahasiswa bimbingan dan panel pengaturan jadwal bimbingan. Tabel mahasiswa menampilkan nama mahasiswa, progres skripsi, serta tombol detail monitoring mahasiswa. Sementara itu, panel jadwal digunakan untuk menentukan tanggal, waktu, metode bimbingan, serta catatan tambahan konsultasi akademik.

Sistem mengambil data mahasiswa bimbingan secara *asynchronous* menggunakan SWR dan Supabase. Potongan kode berikut digunakan agar data mahasiswa bimbingan dapat diperbarui secara otomatis setiap satu menit sehingga kaprodi dan dosen dapat memantau perkembangan akademik mahasiswa secara *real-time*.

```
const { data: cache, isLoading } =
  useSWR(
    'mahasiswa_bimbingan_kaprodi_list',
    fetchMahasiswaBimbinganData,
    {
      revalidateOnFocus: true,
      refreshInterval: 60000
    }
  );
```

Sistem juga melakukan validasi status mahasiswa berdasarkan tahapan skripsi yang sedang dijalani. Kode berikut digunakan untuk menentukan label progres skripsi mahasiswa, seperti proses bimbingan, seminar hasil, perbaikan pasca seminar, hingga sidang skripsi.

```
if (hasSidang) {
  return {
    label: "Sidang Skripsi"
  };
}
```

Sistem juga menyediakan fitur kalender interaktif untuk menentukan jadwal bimbingan mahasiswa. Potongan kode berikut digunakan untuk memilih tanggal pelaksanaan bimbingan melalui kalender yang tersedia pada panel pengaturan jadwal.

```
const handleDateClick =
  (day: number) => {
    setSelectedDate(
      new Date(year, month, day)
    );
  };
};
```

Setelah jadwal dipilih, sistem akan menyimpan data jadwal bimbingan ke tabel *guidance_sessions*. Kode berikut digunakan untuk membuat jadwal bimbingan massal bagi seluruh mahasiswa bimbingan yang dimiliki oleh kaprodi.

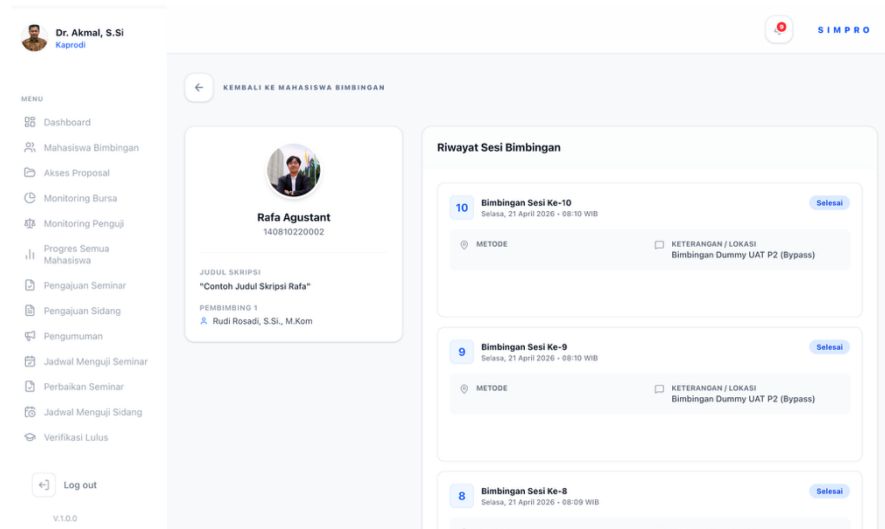
```
const payload =
  students.map((mhs) => ({
    proposal_id: mhs.proposal_id,
    sesi_ke: Number(sesi),
    tanggal: localDate,
    jam: buildTime24(),
    metode,
    status: "belum_dimulai",
    dosen_id: dosenId,
  }));
```

Sistem juga mengirimkan notifikasi otomatis kepada mahasiswa setelah jadwal bimbingan berhasil dibuat. Potongan kode berikut digunakan untuk memberi pemberitahuan kepada mahasiswa mengenai jadwal bimbingan baru yang telah ditetapkan oleh kaprodi melalui sistem SIMPRO.

```
await sendNotification(
  prop.user_id,
  "Jadwal Bimbingan Baru",
  `Dosen ${dosenName}
  telah menetapkan jadwal`
);
```

4.2.14 Halaman Detail Bimbingan Mahasiswa (Kaprodi & Dosen)

Halaman detail bimbingan mahasiswa digunakan oleh kaprodi dan dosen untuk melihat informasi lengkap mahasiswa bimbingan beserta riwayat seluruh sesi konsultasi akademik pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Melalui halaman ini kaprodi dan dosen dapat memantau perkembangan bimbingan, melihat detail setiap sesi, serta melakukan perubahan jadwal apabila diperlukan.



Gambar 4.14 Halaman detail bimbingan mahasiswa

Gambar 4.14 menampilkan tampilan halaman detail bimbingan mahasiswa. Pada halaman ini sistem menampilkan identitas mahasiswa, judul skripsi, nama dosen pembimbing, serta daftar riwayat sesi bimbingan yang telah dilakukan. Setiap sesi menampilkan informasi tanggal, waktu, metode bimbingan, lokasi, dan status pelaksanaan bimbingan.

Sistem mengambil data detail mahasiswa dan riwayat sesi bimbingan secara *asynchronous* menggunakan SWR dan Supabase. Potongan kode berikut digunakan agar data detail mahasiswa dan sesi bimbingan dapat diperbarui secara otomatis tanpa melakukan refresh halaman secara manual.

```
const { data: cache, isLoading }
= useSWR(
  proposalId
  ? `detail_bimbingan_${proposalId}`
  : null,
  () =>
    fetchDetailBimbinganKaprodi(
      proposalId
    )
);
```

Sistem mengambil data riwayat bimbingan berdasarkan proposal mahasiswa dan dosen yang sedang *login*. Kode berikut digunakan untuk menampilkan daftar sesi bimbingan secara terurut mulai dari sesi terbaru hingga sesi terlama.

```
const { data: guidanceData }
= await supabase
  .from("guidance_sessions")
  .select("*")
  .eq("proposal_id", proposalId)
  .eq("dosen_id", user.id)
  .order("sesi_ke", {
    ascending: false
  });
```

Sistem juga menyediakan fitur perubahan jadwal dan status sesi bimbingan melalui modal edit. Potongan kode berikut digunakan untuk memperbarui informasi sesi bimbingan seperti tanggal, waktu, metode, dan status pelaksanaan konsultasi akademik.

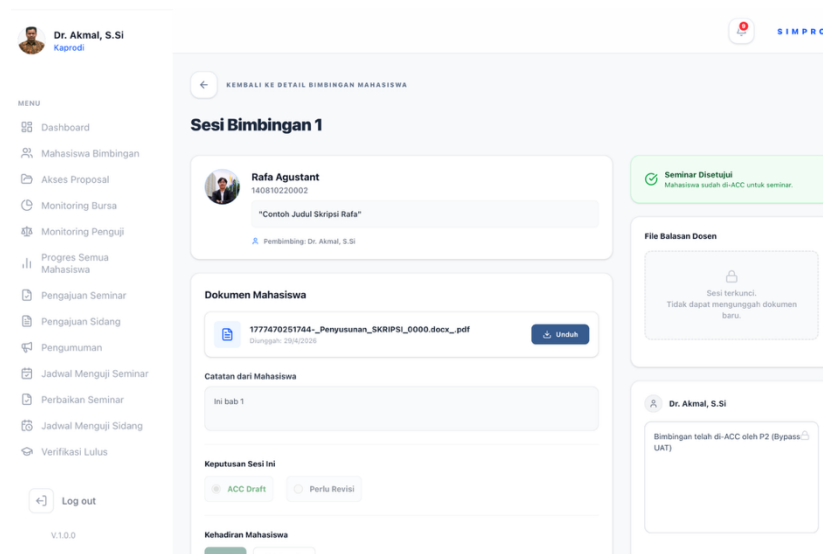
```
await supabase
  .from("guidance_sessions")
  .update({
    tanggal:
      editingSession.tanggal,
    jam: editingSession.jam,
    metode:
      editingSession.metode,
    status:
      editingSession.status,
  })
  .eq("id", editingSession.id);
```

Setelah perubahan jadwal berhasil dilakukan, sistem akan mengirimkan notifikasi otomatis kepada mahasiswa. Kode berikut digunakan untuk memberikan pemberitahuan kepada mahasiswa apabila terjadi perubahan jadwal atau pembatalan sesi bimbingan oleh kaprodi maupun dosen. melalui sistem SIMPRO.

```
await sendNotification(  
    proposal.user_id,  
    title,  
    message  
);
```

4.2.15 Halaman Detail Sesi Bimbingan Mahasiswa (Kaprodi & Dosen)

Halaman detail sesi bimbingan mahasiswa digunakan oleh kaprodi dan dosen untuk melihat dan mengelola detail proses bimbingan mahasiswa pada setiap sesi konsultasi akademik di Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Melalui halaman ini kaprodi maupun dosen dapat melihat draf skripsi mahasiswa, memberikan komentar atau revisi, menentukan status hasil bimbingan, memvalidasi kehadiran mahasiswa, hingga memberikan *ACC* seminar apabila syarat bimbingan telah terpenuhi.



Gambar 4.15 Halaman detail sesi bimbingan mahasiswa

Gambar 4.15 menampilkan tampilan halaman detail sesi bimbingan mahasiswa pada akun kaprodi. Pada halaman ini sistem menampilkan identitas mahasiswa, dokumen draf yang diunggah mahasiswa, catatan mahasiswa, area komentar dosen, status hasil bimbingan, serta fitur unggah file balasan dosen. Sistem juga menyediakan fitur ACC seminar apabila mahasiswa telah memenuhi syarat jumlah bimbingan yang ditentukan.

Sistem mengambil data detail sesi bimbingan secara *asynchronous* menggunakan SWR dan Supabase. Potongan kode berikut digunakan agar data sesi bimbingan dapat diperbarui secara otomatis tanpa perlu memuat ulang halaman secara manual.

```
const { data: cache, isLoading }
= useSWR(
  sessionId
    ? `bimbingan_dosen_${sessionId}`
    : null,
  () =>
    fetchSessionDetail(sessionId)
```

```
);
```

Sistem juga menghitung jumlah bimbingan valid mahasiswa berdasarkan status kehadiran dan hasil revisi sesi bimbingan. Kode berikut digunakan untuk menentukan jumlah sesi bimbingan yang dianggap valid sebagai syarat pengajuan seminar hasil.

```
return (
  s.kehadiran_mahasiswa
    === "hadir" &&
    (
      latest?.status_revisi
        === "disetujui" ||
      latest?.status_revisi
        === "revisi"
    )
);
```

Apabila jumlah bimbingan valid telah mencapai minimal 10 sesi, sistem akan membuka akses *ACC* seminar bagi dosen pembimbing. Potongan kode berikut digunakan untuk memvalidasi bahwa mahasiswa telah memenuhi syarat minimal jumlah bimbingan sebelum memperoleh persetujuan seminar dari dosen pembimbing.

```
validGuidanceCount >= 10
```

Sistem menyediakan fitur penyimpanan hasil bimbingan beserta komentar dan file balasan dosen. Kode berikut digunakan untuk menyimpan komentar, status revisi, serta file balasan dosen ke basis data sistem.

```
await supabase
  .from("session_feedbacks")
  .insert({
    session_id: sessionId,
    komentar,
    file_url: fileUrl,
    status_revisi:
      statusRevisi,
  });
```

Kaprodi dan dosen dapat mengunggah file revisi atau draft balasan untuk mahasiswa melalui Supabase *Storage*. Potongan kode berikut digunakan untuk menyimpan dokumen revisi atau file balasan dosen ke *cloud storage* agar dapat diakses oleh mahasiswa.

```
await supabase.storage
  .from("feedback_draft")
  .upload(
    filePath,
    fileBalasan
  );
```

Sistem menyediakan fitur *ACC* seminar bagi mahasiswa yang telah memenuhi seluruh syarat bimbingan. Kode berikut digunakan untuk mencatat persetujuan seminar mahasiswa oleh dosen pembimbing pada basis data sistem.

```
await supabase
  .from("seminar_requests")
  .update({
    approved_by_p1: true
  })
```

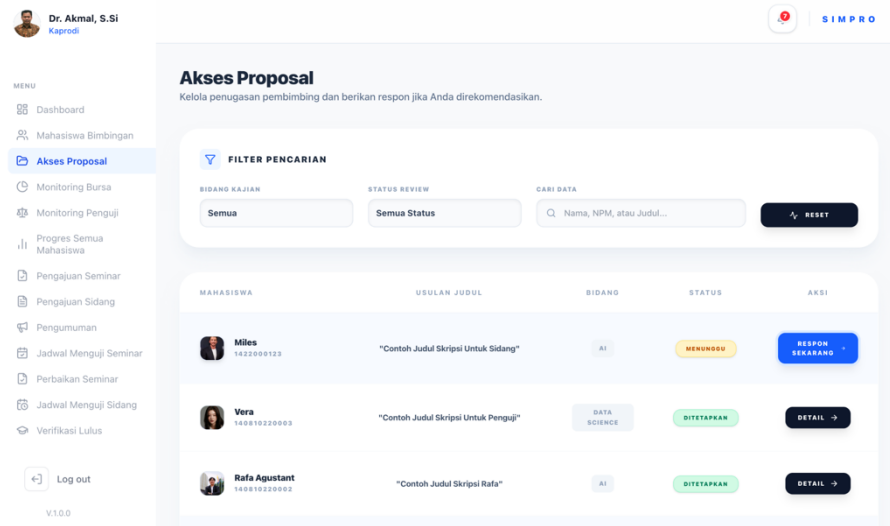
Setelah seminar disetujui, sistem akan mengunci sesi bimbingan sehingga komentar, file balasan, dan hasil revisi tidak dapat diubah kembali. Potongan kode berikut digunakan untuk menjaga konsistensi data bimbingan setelah mahasiswa memperoleh *ACC* seminar dari dosen pembimbing.

```
disabled={isSeminarApproved}
```

4.2.16 Halaman Akses Proposal

Halaman akses proposal digunakan oleh kaprodi dan dosen untuk memantau dan mengelola seluruh usulan proposal skripsi mahasiswa pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Pada halaman ini kaprodi dan dosen dapat

melihat daftar proposal yang diajukan mahasiswa, melakukan pencarian dan filter data, serta memeriksa status rekomendasi dosen pembimbing.



Gambar 4.16 Halaman akses proposal

Gambar 4.16 menampilkan tampilan halaman akses proposal. Sistem menampilkan informasi mahasiswa, judul proposal, bidang kajian, status proposal, serta tombol aksi untuk melihat detail proposal atau memberikan respon terhadap rekomendasi pembimbing.

Sistem mengambil data proposal secara *asynchronous* menggunakan SWR dan Supabase sehingga data proposal dapat diperbarui secara otomatis tanpa perlu memuat ulang halaman. Potongan kode berikut digunakan untuk melakukan pengambilan data proposal secara *real-time* dari basis data.

```
const { data: cache, isLoading }
= useSWR(
  'akses_proposal',
  fetchProposalsData
);
```

Sistem juga memeriksa apakah kaprodi maupun dosen yang sedang *login* termasuk ke dalam daftar rekomendasi dosen pembimbing pada proposal tertentu.

Kode berikut digunakan untuk menentukan apakah proposal perlu segera direspon oleh kaprodi dan dosen sebagai calon dosen pembimbing.

```
const isRecommended =
  proposal_recommendations
    ?.some(
      (r) =>
        r.dosen_id === userId
    );
```

Sistem memeriksa apakah kaprodi atau dosen telah memberikan respon terhadap proposal mahasiswa. Potongan kode berikut digunakan untuk menghindari pemberian respon ganda terhadap proposal yang sama.

```
const alreadyResponded =
  thesis_supervisors
    ?.some(
      (s) =>
        s.dosen_id === userId
    );
```

Sistem melakukan validasi kesiapan dosen pembimbing. Kode berikut digunakan untuk memastikan bahwa proposal telah memperoleh minimal dua persetujuan calon dosen pembimbing sebelum status proposal siap ditetapkan oleh kaprodi.

```
const ready =
  acceptedCount >= 2 &&
  p.status !== "Diterima";
  </main>
);
}
```

Halaman akses proposal juga menyediakan fitur pencarian proposal berdasarkan nama mahasiswa, NPM, maupun judul proposal. Potongan kode berikut digunakan untuk memfilter data proposal sesuai kata kunci pencarian yang dimasukkan pengguna.


```

item.judul
  .toLowerCase()
  .includes(
    searchQuery
      .toLowerCase()
  )

```

Sistem dapat menampilkan tombol aksi yang berubah secara dinamis sesuai status proposal mahasiswa. Potongan kode berikut digunakan untuk memberikan indikator tindakan yang harus dilakukan kaprodi maupun dosen terhadap proposal mahasiswa berdasarkan kondisi proposal saat ini.

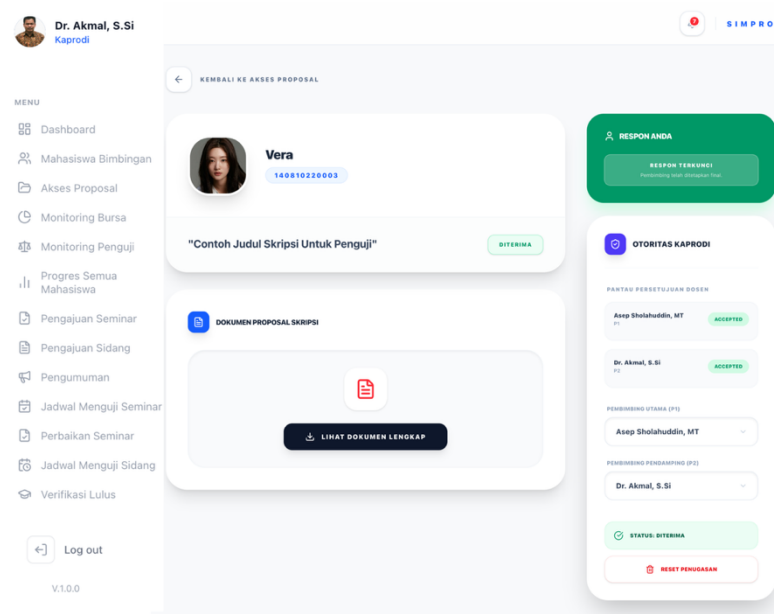
```

showUrgentAction
  ? "RESPON SEKARANG"
  : readyToAssign
  ? "SEGERA TETAPKAN"
  : "DETAIL"

```

4.2.17 Halaman Detail Proposal (Kaprodi)

Halaman detail proposal digunakan oleh kaprodi untuk melihat informasi lengkap proposal skripsi mahasiswa serta mengelola proses penetapan dosen pembimbing pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Pada halaman ini kaprodi dapat melihat identitas mahasiswa, dokumen proposal, status persetujuan dosen pembimbing, hingga melakukan penugasan pembimbing utama dan pembimbing pendamping.



Gambar 4.17 Halaman detail proposal kaprodi

Gambar 4.17 menampilkan tampilan halaman detail proposal. Sistem menampilkan identitas mahasiswa, judul proposal, dokumen proposal skripsi, status proposal, serta panel otoritas kaprodi untuk memantau respon dosen pembimbing dan melakukan penetapan dosen pembimbing final.

Sistem mengambil data detail proposal secara *asynchronous* menggunakan SWR dan Supabase. Potongan kode berikut digunakan untuk mengambil data proposal secara *real-time* tanpa perlu memuat ulang halaman.

```
const { data: cache, isLoading }
= useSWR(
  proposalId
  ? `detail_proposal_${proposalId}`
  : null,
  () =>
    fetchDetailProposalData(
      proposalId
    )
);
```

Sistem menghasilkan *signed URL* untuk menampilkan dokumen proposal yang tersimpan pada Supabase *Storage*. Kode berikut digunakan agar dokumen proposal dapat diakses secara aman dalam jangka waktu tertentu.

```
const { data: signed }
  = await supabase.storage
    .from("proposals")
    .createSignedUrl(
      file_path,
      3600
    );
```

Sistem juga akan memeriksa apakah proposal telah ditetapkan secara *final* oleh kaprodi. Potongan kode berikut digunakan untuk menentukan apakah proses penetapan dosen pembimbing masih dapat dilakukan atau sudah dikunci.

```
const isAssigned =
  !activeStatuses.includes(
    propData.status
  );
```

Sistem menggabungkan data rekomendasi dosen dan data dosen pembimbing yang telah memberikan respon. Kode berikut digunakan untuk menampilkan daftar dosen pembimbing beserta status respon masing-masing dosen terhadap proposal mahasiswa.

```
if (
  !isAssigned &&
  recommendations.length > 0
) {
  displayList.push({
    dosen_id: rec.dosen_id,
    status: 'pending'
  });
}
```

Kaprodi dapat memberikan respon menerima atau menolak proposal mahasiswa melalui sistem. Potongan kode berikut digunakan untuk menyimpan status persetujuan dosen pembimbing terhadap proposal mahasiswa.

```
status:
  isAccepted
    ? "accepted"
    : "rejected"
```

Apabila kaprodi menolak usulan pembimbingan, sistem akan menyimpan alasan penolakan dan hasil penilaian terhadap kelayakan proposal. Potongan kode berikut digunakan untuk menyimpan alasan penolakan dan status kelayakan proposal ke dalam basis data.

```
rejection_reason:
  isAccepted
    ? null
    : rejectReason

is_layak:
  isAccepted
    ? null
    : isLayak
```

Apabila kaprodi menolak usulan pembimbingan dan menilai proposal tidak layak, sistem akan memperbarui status proposal menjadi ditolak dosbing serta membuka kembali akses pengajuan agar mahasiswa dapat melakukan revisi proposal. Potongan kode berikut digunakan untuk mengubah status proposal dan membuka kembali akses pengajuan mahasiswa.

```
await supabase
  .from("proposals")
  .update({
    status:
      "Ditolak Dosbing",
    is_locked: false
  });
```

Kaprodi dapat menetapkan dosen pembimbing utama dan pendamping secara final. Potongan kode berikut digunakan untuk menyimpan data penugasan dosen pembimbing utama dan pembimbing pendamping ke basis data sistem.

```
await supabase
  .from("thesis_supervisors")
  .insert(payload);
```

Setelah penugasan berhasil dilakukan, sistem akan memperbarui status proposal menjadi diterima. Kode berikut digunakan untuk menandai bahwa proposal mahasiswa telah resmi diterima dan dosen pembimbing telah ditetapkan oleh kaprodi.

```
await supabase
  .from("proposals")
  .update({
    status: "Diterima"
  });
```

Sistem akan mengirimkan notifikasi otomatis kepada mahasiswa dan dosen pembimbing setelah penetapan berhasil dilakukan. Potongan kode berikut digunakan untuk memberi pemberitahuan kepada dosen pembimbing dan mahasiswa terkait hasil penetapan.

```
await sendNotification(
  pembimbing1,
  "Penugasan Pembimbing Baru",
  message
);
```

4.2.18 Halaman Monitoring Bursa Dosbing (Kaprodi)

Halaman monitoring bursa dosbing digunakan oleh kaprodi untuk memantau distribusi dosen pembimbing berdasarkan jumlah peminat mahasiswa, jumlah persetujuan dosen pembimbing, serta jumlah mahasiswa bimbingan aktif pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Melalui halaman ini,

kaprodi dapat melihat kondisi kuota bimbingan setiap dosen secara lebih terstruktur sehingga proses pemerataan pembimbing dapat dilakukan dengan lebih efektif dan transparan. Selain itu, halaman ini juga menyediakan fitur pengarsipan data bimbingan, reset semester untuk memulai periode bimbingan baru, serta ekspor rekap data pembimbing ke dalam format Microsoft Excel guna mempermudah proses dokumentasi dan pelaporan akademik.

DOSEN PENGAJAR	PEMINAT (P1/P2)	DISETUJUI	MAHASISWA BIMBINGAN	AKSI
Dr. Akmal, S.Si	P1: 0 P2: 4	2	2	DETAIL →
Asep Sholahuddin, MT	P1: 2 P2: 0	1	1	DETAIL →
Rudi Rosadi, S.Si., M.Kom	P1: 2 P2: 0	1	1	DETAIL →
DR. Juli rejito, m.kom	P1: 0 P2: 0	0	0	DETAIL →

Gambar 4.18 Halaman monitoring bursa dosbing

Gambar 4.18 menampilkan tampilan halaman monitoring bursa dosbing. Sistem menampilkan daftar dosen pembimbing beserta jumlah peminat pembimbing utama (P1), pembimbing pendamping (P2), jumlah mahasiswa yang disetujui, serta jumlah mahasiswa bimbingan aktif.

Sistem mengambil data bursa dosen secara *real-time* menggunakan SWR dan Supabase. Potongan kode berikut digunakan untuk mengambil data monitoring

dosen pembimbing secara realtime berdasarkan mode tampilan yang dipilih pengguna.

```
const { data: bursaData }
= useSWR(
  ['bursa_data', viewModel],
  ([_, mode]) =>
    fetchBursaData(mode)
);
```

Sistem menghitung jumlah mahasiswa yang memilih dosen tertentu sebagai pembimbing utama maupun pembimbing pendamping. Kode berikut digunakan untuk menghitung jumlah mahasiswa yang memilih dosen sebagai pembimbing utama.

```
const p1 =
  recs?.filter(
    r =>
      r.dosen_id === d.id &&
      r.tipe === "pembimbing1"
  ).length || 0;
```

Sistem juga menghitung jumlah mahasiswa aktif yang masih berada dalam proses bimbingan. Potongan kode berikut digunakan untuk memfilter mahasiswa yang masih aktif menjalani proses skripsi dan belum dinyatakan lulus.

```
const mahasiswaAktif =
  bimbinganDosen.filter(
    s =>
      s.proposal?.status
      === "Diterima"
  );
```

Sistem menyediakan fitur pencarian dosen berdasarkan nama dosen pembimbing. Kode berikut digunakan untuk menampilkan data dosen sesuai kata kunci pencarian yang dimasukkan oleh kaprodi.

```
const filtered =
  stats.filter(
    s =>
      s.nama
        .toLowerCase()
        .includes(
          searchTerm.toLowerCase()
        )
  );
```

Sistem menyediakan fitur penyimpanan arsip data monitoring bimbingan setiap semester. Potongan kode berikut digunakan untuk menyimpan data monitoring dosen pembimbing ke tabel arsip semester.

```
await supabase
  .from("bimbingan_archives")
  .upsert(payload);
```

Selain penyimpanan arsip, sistem menyediakan fitur *reset* semester untuk menghapus data pembimbing dan antrean peminat dosen pada semester berjalan. Kode berikut digunakan untuk menghapus data dosen pembimbing aktif ketika proses reset semester dilakukan oleh kaprodi.

```
await supabase
  .from(
    'thesis_supervisors'
  )
  .delete();
```

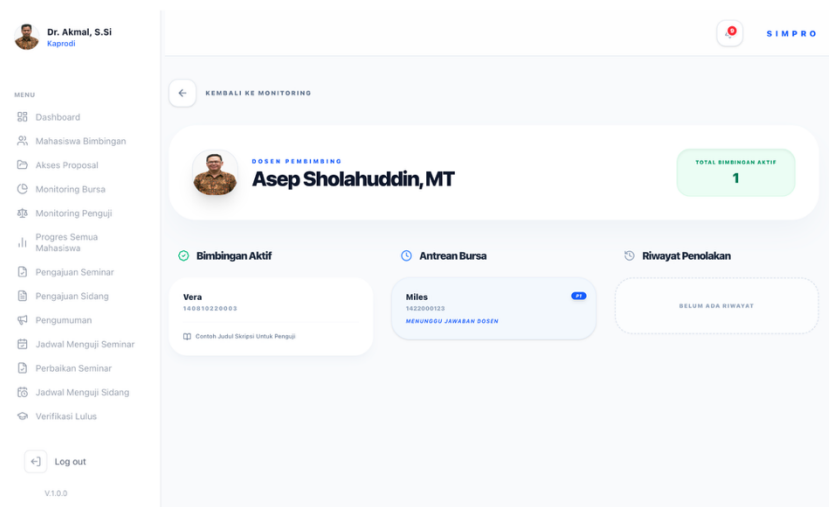
Sistem juga menyediakan fitur ekspor rekap data pembimbing ke dalam format Microsoft Excel. Potongan kode berikut digunakan untuk menghasilkan file rekap pembimbing dalam format .xlsx sehingga dapat diunduh dan digunakan sebagai laporan administrasi akademik.

```
XLSX.writeFile(
  workbook,
  fileName
);
```

4.2.19 Halaman Detail Monitoring Bursa Dosbing (Kaprodi)

Halaman detail monitoring bursa dosbing digunakan oleh kaprodi untuk melihat informasi detail aktivitas dosen pembimbing pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Pada halaman ini kaprodi dapat memantau jumlah mahasiswa bimbingan aktif, antrean mahasiswa pada bursa dosen pembimbing, serta riwayat penolakan proposal oleh dosen pembimbing.

Gambar 4.19 menampilkan tampilan halaman detail monitoring bursa dosbing. Sistem menampilkan identitas dosen pembimbing, total mahasiswa bimbingan aktif, daftar mahasiswa yang sedang dibimbing, antrean mahasiswa yang masih menunggu respon dosen atau persetujuan kaprodi, serta riwayat penolakan proposal beserta alasan penolakannya.



Gambar 4.19 Halaman detail monitoring bursa dosbing

Sistem mengambil data detail monitoring dosen secara *real-time* menggunakan SWR dan Supabase. Potongan kode berikut digunakan untuk

mengambil data monitoring dosen pembimbing secara *real-time* berdasarkan dosen yang dipilih.

```
const { data: cache }
= useSWR(
  dosenId
  ? `detail_bursa_${dosenId}`
  : null,
  () =>
    fetchDetailBursaData(
      dosenId
    )
);
```

Sistem mengambil data mahasiswa bimbingan dari tabel *thesis_supervisors* beserta data proposal mahasiswa. Kode berikut digunakan untuk mengambil data hubungan dosen pembimbing dan mahasiswa.

```
const { data: supsData }
= await supabase
  .from(
    "thesis_supervisors"
  )
  .select(`
    status,
    role,
    proposal:proposals (
      judul,
      status
    )
  `);
```

Sistem memfilter mahasiswa yang masih aktif menjalani proses bimbingan. Potongan kode berikut digunakan untuk memastikan hanya mahasiswa yang belum lulus yang ditampilkan pada daftar bimbingan aktif.

```
const bimbinganAktif =
  supervisors.filter(
    s => {
      const isLulus =
        p?.status_lulus === true;
      return !isLulus;
    }
  );
```

Sistem juga menampilkan antrean mahasiswa pada bursa dosen pembimbing. Kode berikut digunakan untuk menampilkan mahasiswa yang masih menunggu respon dosen pembimbing terhadap usulan proposalnya.

```
const antreanBelumDijawab =
  peminat.filter(
    p =>
      !processedProposalIds
        .includes(
          p.proposal_id
        )
  );
```

Sistem menggabungkan antrean mahasiswa yang telah disetujui dosen namun masih menunggu penetapan kaprodi. Potongan kode berikut digunakan untuk mengelompokkan seluruh antrean proposal yang masih berada pada proses bursa dosen pembimbing.

```
const antreanGabungan = [
  ...antreanDisetujuiDosen,
  ...antreanBelumDijawab
];
```

Selain mahasiswa aktif dan antrean bursa, sistem juga menampilkan riwayat penolakan proposal oleh dosen pembimbing. Kode berikut digunakan untuk menampilkan daftar proposal mahasiswa yang pernah ditolak oleh dosen pembimbing.

```
const riwayatKeputusan =
  supervisors.filter(
    s =>
      s.status === "rejected"
  );
```

Sistem juga menampilkan alasan penolakan proposal yang diberikan oleh dosen pembimbing. Potongan kode berikut digunakan untuk menampilkan alasan penolakan proposal apabila dosen memberikan catatan penolakan pada sistem.

```
s.rejection_reason ||
```

" Tidak ada alasan spesifik "

4.2.20 Halaman Monitoring Beban Penguji (Kaprodi)

Halaman monitoring beban penguji digunakan oleh kaprodi untuk memantau distribusi tugas dosen penguji pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Pada halaman ini kaprodi dapat melihat jumlah beban menguji setiap dosen, status distribusi penguji, melakukan pencarian data dosen, serta mengunduh rekapitulasi beban penguji dalam format Excel.

DOSEN	BEBAN MENGUJI	STATUS DISTRIBUSI	AKSI
DR. Juli rejito, m.kom 1211	2	SUDAH DITUGASKAN	DETAIL →
Gerald Andreas .	2	SUDAH DITUGASKAN	DETAIL →
Asap Sholahuddin, MT 140510	1	SUDAH DITUGASKAN	DETAIL →
Rudi Rosadi, S.Si., M.Kom .	1	SUDAH DITUGASKAN	DETAIL →
Dr. Akmal, S.Si 121212121212	0	BELUM DITUGASKAN	DETAIL →

Menampilkan 1 - 5 dari 5 dosen

Gambar 4.20 Halaman monitoring beban penguji

Gambar 4.20 menampilkan tampilan halaman monitoring beban penguji. Sistem menampilkan daftar dosen beserta jumlah tugas menguji yang sedang aktif. Data tersebut digunakan kaprodi untuk memastikan distribusi tugas penguji tetap merata dan tidak menumpuk pada dosen tertentu.

Sistem mengambil data monitoring penguji secara *real-time* menggunakan SWR dan Supabase. Potongan kode berikut digunakan untuk mengambil data distribusi beban penguji secara *real-time*.

```
const { data: stats = [] }
  = useSWR(
    'monitoring_penguji_list',
    fetchPengujiData
  );
```

Sistem mengambil data seluruh dosen dan kaprodi dari tabel *profiles*. Kode tersebut digunakan untuk mengambil data identitas dosen yang akan dimonitor pada sistem.

```
const { data: dosens }
  = await supabase
    .from("profiles")
    .select(
      "id, nama, nip"
    )
    .in(
      "role",
      ["dosen", "kaprodi"]
    );
```

Sistem juga mengambil data penugasan penguji yang masih aktif. Potongan kode berikut digunakan untuk memastikan perhitungan beban menguji hanya dilakukan pada mahasiswa yang belum dinyatakan lulus.

```
const activeAssignments =
  assignments.filter(a => {
    const isLulus =
      prop?.status_lulus === true;
    return !isLulus;
  });
```

Jumlah beban menguji setiap dosen dihitung berdasarkan jumlah penugasan aktif yang dimiliki dosen tersebut. Kode berikut digunakan untuk menghitung total tugas menguji aktif pada masing-masing dosen.

```
const count =
  activeAssignments.filter(
    a =>
```

```
a.dosen_id === d.id
).length;
```

Sistem kemudian menentukan status distribusi pengujian berdasarkan jumlah penugasan yang dimiliki dosen. Potongan kode berikut digunakan untuk memberikan informasi status distribusi tugas pengujian pada sistem.

```
status_tugas:
  count > 0
    ? "Sudah Ditugaskan"
    : "Belum Ditugaskan"
```

Sistem menyediakan fitur pencarian dosen berdasarkan nama untuk memudahkan pencarian. Kode berikut digunakan untuk mempermudah kaprodi dalam mencari data dosen pengujian tertentu.

```
const filtered =
  stats.filter(s =>
    s.nama
      .toLowerCase()
      .includes(
        searchTerm.toLowerCase()
      )
  );
```

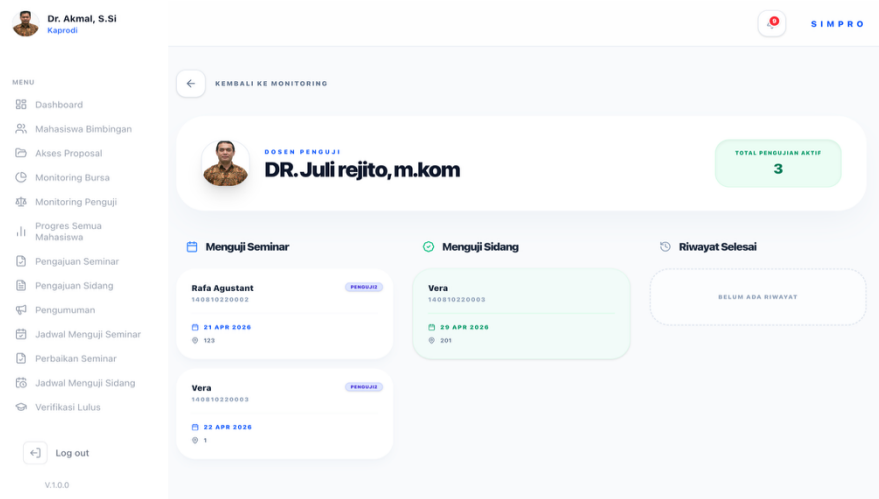
Sistem juga menyediakan fitur *export* data monitoring ke dalam format Excel. Potongan kode berikut digunakan untuk menghasilkan file rekapitulasi beban pengujian dalam format .xlsx yang dapat diunduh oleh kaprodi.

```
XLSX.writeFile(
  workbook,
  `Rekap_Beban_Pengujian.xlsx`
);
```

4.2.21 Halaman Detail Monitoring Beban Pengujian (Kaprodi)

Halaman detail monitoring beban pengujian digunakan oleh kaprodi untuk melihat rincian aktivitas dosen pengujian pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Pada halaman ini kaprodi dapat memantau daftar mahasiswa

yang sedang diuji pada tahap seminar maupun sidang akhir, melihat jadwal pengujian, serta memantau riwayat mahasiswa yang telah selesai diuji.



Gambar 4.21 Halaman detail monitoring beban penguji

Gambar 4.21 menampilkan tampilan halaman detail monitoring beban penguji. Sistem menampilkan identitas dosen penguji, total pengujian aktif, daftar mahasiswa yang sedang menjalani seminar maupun sidang akhir, serta riwayat mahasiswa yang telah menyelesaikan proses sidang skripsi.

Sistem mengambil data detail penguji secara *real-time* menggunakan SWR dan Supabase. Potongan kode berikut digunakan untuk mengambil data aktivitas dosen penguji berdasarkan dosen yang dipilih.

```
const { data: cache }
= useSWR(
  dosenId
  ? `detail_penguji_${dosenId}`
  : null,
  () =>
    fetchDetailPengujiData(
      dosenId
    )
)
```

```
);
```

Sistem mengambil data jadwal seminar yang diikuti dosen penguji melalui tabel *examiners*. Kode berikut digunakan untuk mengambil data seminar skripsi yang sedang diujikan oleh dosen penguji.

```
const { data: seminarData }
= await supabase
  .from("examiners")
  .select(`
    role,
    seminar_request:
    seminar_requests (
      schedule:
      seminar_schedules (
        tanggal,
        ruangan
      )
    )
  `);
```

Selain seminar, sistem juga mengambil data sidang akhir yang diikuti dosen penguji. Potongan kode berikut digunakan untuk mengambil data sidang akhir mahasiswa yang diuji oleh dosen.

```
const { data: sidangData }
= await supabase
  .from("sidang_grades")
  .select(`
    nilai_akhir,
    sidang_request:
    sidang_requests (
      tanggal_sidang,
      ruangan
    )
  `);
```

Sistem kemudian memfilter mahasiswa yang masih aktif menjalani proses seminar maupun sidang. Kode berikut digunakan untuk memastikan hanya mahasiswa yang belum lulus yang ditampilkan pada daftar pengujian aktif.

```
const antreanSeminar =
  seminars.filter(s => {
    const isLulus =
      p?.status_lulus === true;
    return !isLulus;
```



```
});
```

Sistem menghitung total beban pengujian aktif yang dimiliki dosen penguji.

Potongan kode berikut digunakan untuk menghitung total mahasiswa yang sedang diuji oleh dosen pada tahap seminar maupun sidang akhir.

```
const totalAktif =
  antreanSeminar.length +
  antreanSidang.length;
```

Selain pengujian aktif, sistem juga menampilkan riwayat mahasiswa yang telah selesai diuji. Kode berikut digunakan untuk menggabungkan dan menghapus data duplikat riwayat mahasiswa yang telah selesai menjalani seminar atau sidang akhir.

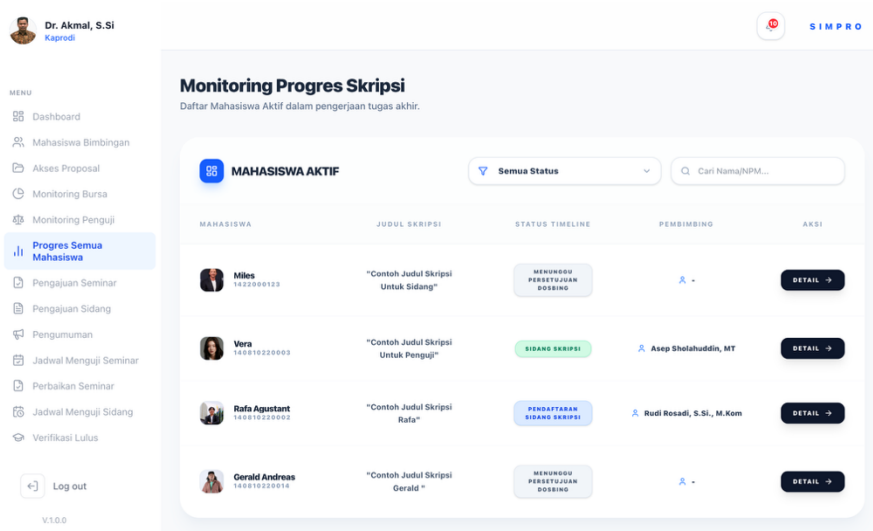
```
const riwayatSelesai =
  Array.from(
    new Map(
      riwayatRaw.map(
        item => [
          item.mahasiswa?.npm,
          item
        ]
      )
    ).values()
  );
```

Sistem menyediakan fungsi format tanggal agar jadwal seminar dan sidang lebih mudah dibaca oleh pengguna. Potongan kode berikut digunakan untuk mengubah format tanggal menjadi format lokal Indonesia sebelum ditampilkan pada antarmuka sistem.

```
getFormattedDate(
  sched?.tanggal
)
```

4.2.22 Halaman Progres Semua Mahasiswa (Kaprod)

Halaman progres semua mahasiswa digunakan oleh kaprod untuk memantau perkembangan tahapan skripsi seluruh mahasiswa aktif pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Pada halaman ini kaprod dapat melihat status timeline skripsi mahasiswa mulai dari pengajuan proposal, proses bimbingan, seminar hasil, perbaikan pasca seminar, hingga sidang skripsi akhir.



Gambar 4.22 Halaman progres semua mahasiswa

Gambar 4.22 menampilkan tampilan halaman progres semua mahasiswa. Sistem menampilkan daftar mahasiswa aktif beserta judul skripsi, status *timeline* skripsi, dosen pembimbing utama, serta tombol detail monitoring mahasiswa.

Sistem mengambil data progres mahasiswa secara *real-time* menggunakan SWR dan Supabase. Potongan kode berikut digunakan untuk mengambil data progres seluruh mahasiswa aktif secara *real-time*.

```
const { data = [] }
= useSWR(
  'progres_semua_mahasiswa_kaprodi',
  fetchProgresSemuaMahasiswa
);
```

Sistem mengambil data proposal mahasiswa beserta relasi data seminar, sidang, bimbingan, dan dosen pembimbing. Kode berikut digunakan untuk mengambil seluruh data akademik mahasiswa yang dibutuhkan dalam proses monitoring progres skripsi.

```
const { data: proposals }
= await supabase
  .from("proposals")
  .select(`
    judul,
    status,
    seminar_requests,
    sidang_requests,
    guidance_sessions
  `);
```

Sistem menghitung jumlah bimbingan valid mahasiswa berdasarkan status kehadiran dan hasil revisi bimbingan. Potongan kode berikut digunakan untuk menentukan jumlah sesi bimbingan valid yang dimiliki mahasiswa.

```
const count =
  sessions.filter(
    s =>
      s.kehadiran_mahasiswa
        === 'hadir' &&
      s.session_feedbacks?.[0]
        ?.status_revisi
          === "disetujui"
  ).length;
```

Selain itu, sistem melakukan validasi kelayakan seminar mahasiswa berdasarkan jumlah bimbingan dan persetujuan dosen pembimbing. Kode berikut digunakan untuk menentukan apakah mahasiswa telah memenuhi syarat mengikuti seminar hasil.

```
let isEligible =
  p1Count >= 10 &&
```

```
p2Count >= 10 &&
approvedByAll;
```

Sistem kemudian memetakan status akademik mahasiswa ke dalam tampilan timeline skripsi. Potongan kode berikut digunakan untuk mengubah data status akademik menjadi label *timeline* yang ditampilkan pada antarmuka sistem.

```
const ui =
  mapStatusToUI({
    proposalStatus: p.status,
    hasSeminar:
      !!activeSeminarReq,
    hasSidang:
      hasSidang,
  });
```

Sistem juga melakukan sinkronisasi status progres mahasiswa berdasarkan kondisi seminar, revisi, dan sidang skripsi. Kode berikut digunakan untuk memperbarui tahapan akademik mahasiswa secara dinamis sesuai perkembangan proses skripsi mahasiswa.

```
if (hasSidang) {
  finalTahap =
    "Sidang Skripsi";
}
```

Halaman progres semua mahasiswa menyediakan fitur filter status progres mahasiswa. Potongan kode berikut digunakan untuk menampilkan data mahasiswa berdasarkan kategori status tertentu.

```
const matchStatus =
  filterStatus
  === "Semua Status" ||
  mhs.status
  === filterStatus;
```

Selain filter status, sistem juga menyediakan fitur pencarian mahasiswa berdasarkan nama, NPM, maupun judul skripsi. Kode berikut digunakan untuk mempermudah kaprodi dalam mencari data mahasiswa tertentu pada sistem monitoring progres skripsi.

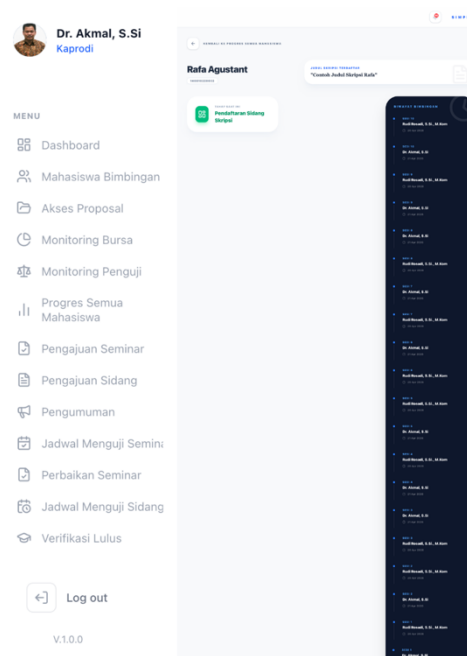
```

mhs.nama
  .toLowerCase()
  .includes(
    searchTerm
      .toLowerCase()
  )

```

4.2.23 Halaman Detail Progres (Kaprodi)

Halaman detail progres digunakan oleh kaprodi untuk memantau perkembangan tahapan skripsi seorang mahasiswa secara lebih rinci. Pada halaman ini kaprodi dapat melihat identitas mahasiswa, judul skripsi, tahapan akademik saat ini, serta riwayat sesi bimbingan yang telah dilakukan mahasiswa bersama dosen pembimbing.



Gambar 4.23 Halaman detail progres

Gambar 4.23 menampilkan tampilan halaman detail progres mahasiswa. Sistem menampilkan informasi progres skripsi mahasiswa beserta timeline riwayat bimbingan yang tersusun berdasarkan sesi bimbingan terbaru.

Sistem mengambil detail progres mahasiswa secara *real-time* menggunakan SWR dan Supabase. Potongan kode berikut digunakan untuk mengambil data detail progres mahasiswa berdasarkan ID proposal secara *real-time*.

```
const { data: cache }
= useSWR(
  `detail_progres_kaprodi_${proposalId}`,
  () => fetchDetailProgresData(proposalId)
);
```

Sistem mengambil data proposal mahasiswa beserta relasi data seminar, sidang, dan bimbingan. Kode berikut digunakan untuk memperoleh seluruh data akademik mahasiswa yang dibutuhkan pada halaman detail progres.

```
const { data: propData }
= await supabase
  .from("proposals")
  .select(`
    judul,
    seminar_requests,
    thesis_supervisors,
    guidance_sessions
  `)
  .single();
```

Sistem melakukan validasi sesi bimbingan berdasarkan kehadiran mahasiswa dan status revisi dosen pembimbing. Potongan kode berikut digunakan untuk memastikan bahwa hanya sesi bimbingan valid yang dihitung pada sistem.

```
const validSessions =
  sessions.filter((s) => {
    return (
      s.kehadiran_mahasiswa
        === 'hadir'
    );
  });
```

Sistem juga melakukan perhitungan jumlah bimbingan pada masing-masing dosen pembimbing. Kode berikut digunakan untuk memisahkan jumlah sesi bimbingan pembimbing utama dan *co*-pembimbing.

```
if (
  sp.role === "utama"
) p1Count = count;
else p2Count = count;
```

Selain itu, sistem melakukan validasi kelayakan seminar mahasiswa berdasarkan jumlah bimbingan dan persetujuan dosen pembimbing. Potongan kode berikut digunakan untuk menentukan apakah mahasiswa telah memenuhi syarat mengikuti seminar hasil.

```
let isEligible =
  p1Count >= 10 &&
  p2Count >= 10 &&
  approvedByAll;
```

Sistem memetakan status akademik mahasiswa ke dalam tahapan *timeline* skripsi. Kode berikut digunakan untuk menghasilkan label tahapan akademik mahasiswa pada sistem monitoring progres.

```
const ui =
  mapStatusToUI({
    proposalStatus:
      propData.status,
    hasSeminar:
      hasSeminar,
    hasSidang:
      hasSidangFound,
  });
```

Sistem juga melakukan sinkronisasi status akhir mahasiswa berdasarkan proses seminar dan sidang skripsi. Potongan kode berikut digunakan untuk memperbarui status tahapan skripsi mahasiswa secara dinamis sesuai perkembangan akademik mahasiswa.

```

if (sidangData) {
    finalTahap =
        "Sidang Skripsi";
}

```

Sistem menampilkan riwayat sesi bimbingan mahasiswa berdasarkan sesi yang telah disetujui dosen pembimbing. Kode berikut digunakan untuk menampilkan riwayat bimbingan yang valid pada *sidebar timeline* bimbingan mahasiswa.

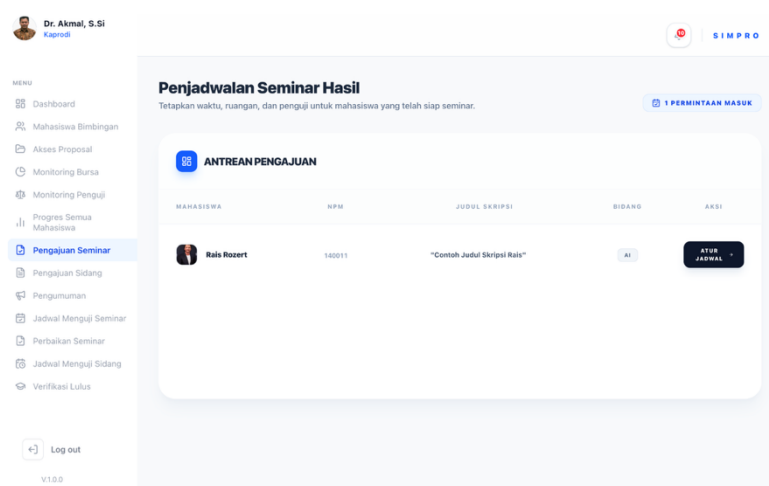
```

.in(
    "session_feedbacks.status_revisi",
    ["disetujui", "revisi"]
)

```

4.2.24 Halaman Pengajuan Seminar (Kaprodi)

Halaman pengajuan seminar digunakan oleh ketua program studi untuk melihat daftar pengajuan seminar mahasiswa yang telah memenuhi persyaratan seminar. Pada halaman ini kaprodi dapat melihat informasi mahasiswa, judul skripsi, bidang penelitian, serta melakukan penjadwalan seminar hasil mahasiswa.



Gambar 4.24 Halaman pengajuan seminar

Gambar 4.24 menampilkan tampilan halaman pengajuan seminar. Sistem menampilkan antrean pengajuan seminar mahasiswa yang telah disetujui oleh dosen pembimbing dan siap dijadwalkan seminar hasil.

Sistem menggunakan mekanisme SWR untuk melakukan pengambilan data secara otomatis dan memperbarui data pengajuan seminar secara *real-time*. Potongan kode berikut digunakan untuk melakukan sinkronisasi data pengajuan seminar.

```
const {
  data: students = [],
  isLoading
} = useSWR(
  "pengajuan_seminar_kaprodi_list",
  fetchSeminarRequests,
  {
    revalidateOnFocus: true,
    refreshInterval: 60000
  }
);
```

Sistem mengambil data pengajuan seminar mahasiswa dari *database* berdasarkan status pengajuan seminar yang telah disetujui oleh dosen pembimbing. Potongan kode berikut digunakan untuk memperoleh data pengajuan seminar mahasiswa beserta informasi proposal dan profil mahasiswa.

```
const { data, error } =
  await supabase
    .from(
      "seminar_requests"
    )
    .select(`
      id,
      status,
      proposal:proposals (
        judul,
        bidang,
        user:profiles (
          nama,
```

```

        npm,
        avatar_url
      )
    )
  `)
  .eq(
    "status",
    "Menunggu Persetujuan"
  )
  .eq(
    "approved_by_p1",
    true
  )
  .eq(
    "approved_by_p2",
    true
  )
  .order(
    "created_at",
    {
      ascending: false
    }
  )
);

```

Sistem menampilkan jumlah pengajuan seminar yang masuk pada halaman kaprodi. Potongan kode berikut digunakan untuk menampilkan total permintaan pengajuan seminar.

```

{students.length}
Permintaan Masuk

```

Sistem menampilkan data mahasiswa yang mengajukan seminar ke dalam bentuk tabel. Potongan kode berikut digunakan untuk menampilkan informasi mahasiswa pada tabel pengajuan seminar.

```

students.map((item) => {
  const userData =
    item.proposal?.user;
});

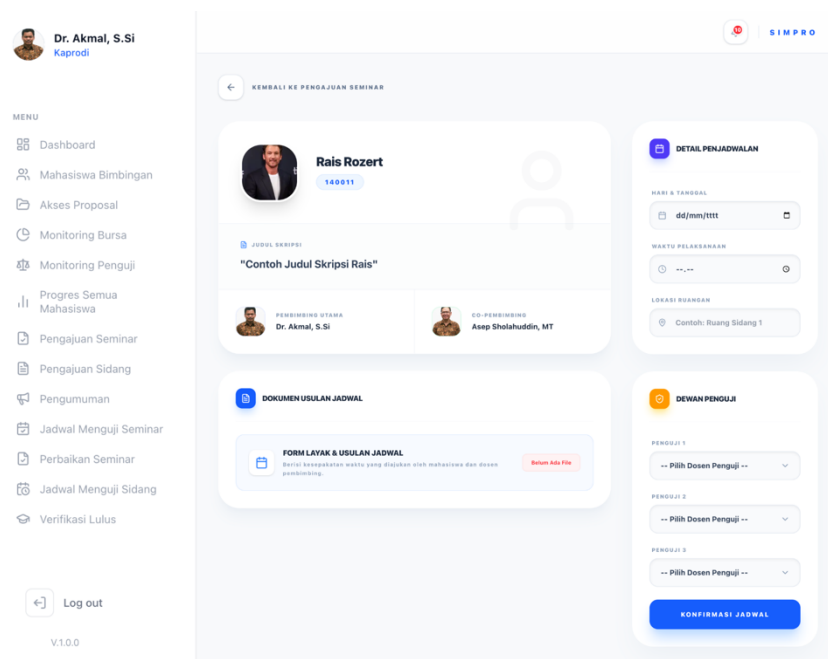
```

4.2.25 Halaman Penjadwalan Seminar (Kaprodi)

Halaman penjadwalan seminar digunakan oleh ketua program studi untuk menetapkan jadwal pelaksanaan seminar mahasiswa. Pada halaman ini kaprodi

dapat melihat informasi mahasiswa, judul skripsi, dosen pembimbing, dokumen usulan jadwal seminar, menentukan waktu pelaksanaan seminar, lokasi ruangan, serta menetapkan dewan penguji seminar.

Gambar 4.25 menampilkan tampilan halaman penjadwalan seminar. Sistem menampilkan detail pengajuan seminar mahasiswa beserta fitur penjadwalan seminar dan penentuan dosen penguji.



Gambar 4.25 Halaman penjadwalan seminar

Sistem mengambil data pengajuan seminar mahasiswa berdasarkan id pengajuan seminar yang dipilih. Potongan kode berikut digunakan untuk memperoleh data proposal mahasiswa beserta data pembimbing dan dokumen seminar.

```
const {
  data: requestData
} = await supabase
  .from(
    "seminar_requests"
  )
```

```

.select(`
  id,
  proposal:proposals (
    id,
    judul,
    file_path,
    user_id,
    user:profiles(
      nama,
      npm,
      avatar_url
    ),
    supervisors:
    thesis_supervisors(
      role,
      dosen_id,
      dosen:profiles(
        nama,
        avatar_url
      )
    ),
    docs:
    seminar_documents(
      nama_dokumen,
      file_url
    )
  )
`)
.eq(
  "id",
  requestId
)
.single();

```

Sistem mengambil daftar dosen yang dapat dipilih sebagai dosen penguji seminar. Potongan kode berikut digunakan untuk memperoleh data dosen dan kaprodi dari *database*.

```

const {
  data: dosenData
} = await supabase
  .from("profiles")
  .select("id, nama")
  .in("role", [
    "dosen",
    "kaprodi"
  ])
  .order("nama");

```

Sistem menyediakan fitur untuk mengakses dokumen usulan jadwal seminar mahasiswa secara aman menggunakan *signed URL*. Potongan kode berikut digunakan untuk menghasilkan URL sementara dokumen seminar mahasiswa.

```
const { data: signedDoc } =
  await supabase.storage
    .from("docseminar")
    .createSignedUrl(
      formJadwal.file_url,
      3600
    );
```

Sistem menyimpan data jadwal seminar mahasiswa ke *database*. Potongan kode berikut digunakan untuk menyimpan tanggal, jam, dan ruangan seminar mahasiswa.

```
await supabase
  .from(
    "seminar_schedules"
  )
  .upsert({
    seminar_request_id:
      requestId,
    tanggal: tanggal,
    jam: jam,
    ruangan: ruangan,
  }, {
    onConflict:
      "seminar_request_id"
  });
```

Sistem menyimpan data dosen penguji seminar mahasiswa. Potongan kode berikut digunakan untuk menyimpan dosen penguji seminar ke *database*.

```

await supabase
  .from("examiners")
  .insert([
    {
      seminar_request_id:
        requestId,
      dosen_id:
        penguji1,
      role:
        "penguji1"
    },
    {
      seminar_request_id:
        requestId,
      dosen_id:
        penguji2,
      role:
        "penguji2"
    },
    {
      seminar_request_id:
        requestId,
      dosen_id:
        penguji3,
      role:
        "penguji3"
    }
  ]);

```

Sistem mengirimkan notifikasi otomatis kepada mahasiswa, dosen pembimbing, dan dosen penguji setelah jadwal seminar berhasil ditetapkan. Potongan kode berikut digunakan untuk mengirimkan notifikasi seminar kepada seluruh pihak terkait.

```

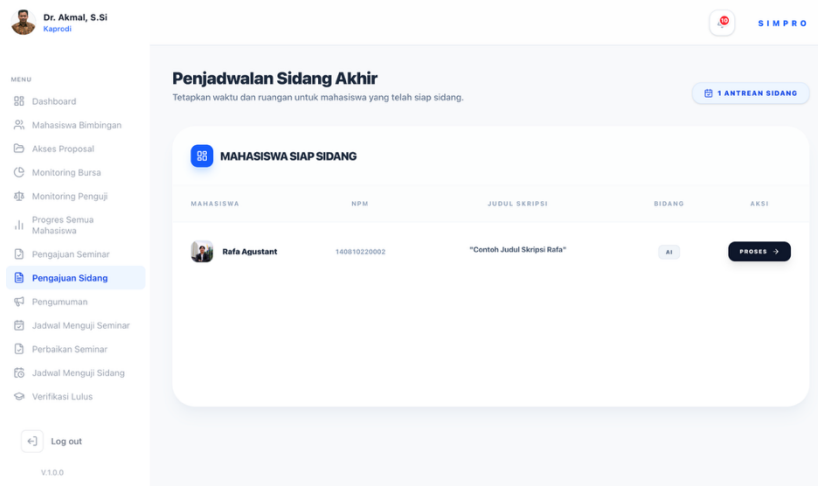
await Promise.all(
  notificationPromises
);

```

4.2.26 Halaman Pengajuan Sidang (Kaprod)

Halaman pengajuan sidang digunakan oleh kaprod untuk melihat daftar mahasiswa yang telah memenuhi persyaratan sidang skripsi akhir pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Pada halaman ini kaprod dapat

memantau antrean mahasiswa siap sidang, melihat informasi akademik mahasiswa, serta melanjutkan proses penjadwalan sidang skripsi.



Gambar 4.26 Halaman pengajuan sidang

Gambar 4.26 menampilkan tampilan halaman pengajuan sidang. Sistem menampilkan daftar mahasiswa yang telah lolos tahap seminar dan dinyatakan siap mengikuti sidang skripsi akhir.

Sistem mengambil data pengajuan sidang mahasiswa secara *real-time* menggunakan SWR dan Supabase. Potongan kode berikut digunakan untuk mengambil daftar mahasiswa yang berada pada antrean sidang skripsi.

```
const { data: students = [] }
= useSWR(
  'pengajuan_sidang_kaprodi_list',
  fetchSidangRequests
);
```

Sistem mengambil data mahasiswa yang memiliki status pengajuan sidang “menunggu penjadwalan”. Kode berikut digunakan untuk menampilkan mahasiswa yang telah siap dijadwalkan sidang oleh kaprodi.

```
const { data }
= await supabase
  .from("sidang_requests")
  .select(`
    status,
    proposal:proposals (
      judul,
      bidang
    )
  `)
  .in(
    'status',
    ['menunggu_penjadwalan']
  );
```

Sistem juga mengambil data identitas mahasiswa melalui relasi tabel proposal dan profil pengguna. Potongan kode berikut digunakan untuk menampilkan identitas mahasiswa pada tabel pengajuan sidang.

```
user:profiles (
  nama,
  npm,
  avatar_url
)
```

Data hasil query kemudian dipetakan ke dalam struktur antarmuka sistem. Kode berikut digunakan untuk membentuk data mahasiswa yang akan ditampilkan pada tabel pengajuan sidang.

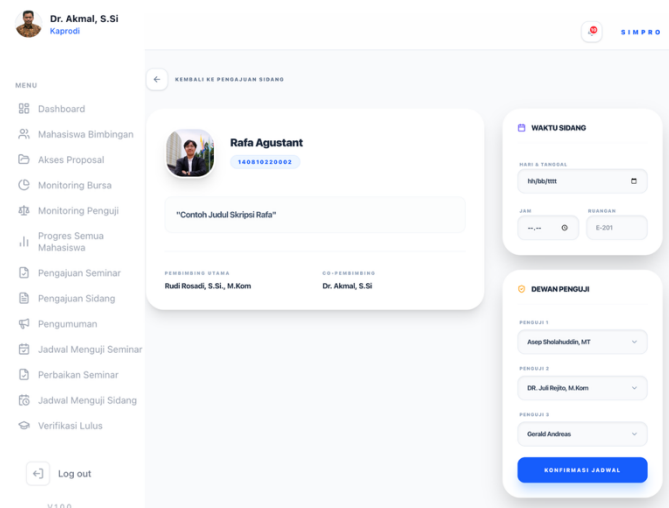
```
return {
  nama: profile.nama,
  npm: profile.npm,
  judul: prop.judul,
  bidang: prop.bidang,
};
```


Sistem menyediakan tombol proses untuk melanjutkan penjadwalan sidang mahasiswa. Kode berikut digunakan untuk mengarahkan kaprodi menuju halaman penjadwalan sidang mahasiswa yang dipilih.

```
<Link
  href={` /kaprodi/
  penjadwalansidang`}
>
```

4.2.27 Halaman Penjadwalan Sidang (Kaprodi)

Halaman penjadwalan sidang digunakan oleh kaprodi untuk menetapkan jadwal pelaksanaan sidang skripsi mahasiswa pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Pada halaman ini kaprodi dapat menentukan tanggal sidang, jam sidang, ruangan, serta menetapkan dewan penguji untuk mahasiswa yang telah dinyatakan siap sidang.



Gambar 4.27 Halaman penjadwalan sidang

Gambar 4.27 menampilkan tampilan halaman penjadwalan sidang. Sistem menampilkan identitas mahasiswa, judul skripsi, dosen pembimbing, dokumen skripsi final, form penjadwalan sidang, serta pemilihan dewan penguji.

Sistem mengambil data penjadwalan sidang secara *real-time* menggunakan SWR dan Supabase. Potongan kode berikut digunakan untuk mengambil data mahasiswa dan data penjadwalan sidang secara *real-time*.

```
const { data: cache }
= useSWR(
  `penjadwalan_sidang_${seminarRequestId}`,
  () =>
    fetchJadwalSidangData(
      seminarRequestId
    )
);
```

Sistem mengambil data seminar dan proposal mahasiswa beserta data dosen pembimbing. Kode berikut digunakan untuk memperoleh data akademik mahasiswa yang akan dijadwalkan sidang.

```
const { data: seminar }
= await supabase
  .from("seminar_requests")
  .select(`
    proposal:proposals (
      judul,
      user:profiles (
        nama,
        npm
      ),
      supervisors:
        thesis_supervisors (
          role
        )
      )
  `);
```

Sistem mengambil dokumen skripsi final mahasiswa yang telah diverifikasi. Potongan kode berikut digunakan untuk memastikan hanya dokumen skripsi *final* yang telah diverifikasi yang dapat digunakan pada proses sidang.

```
.eq(
  "nama_dokumen",
  "file_skripsi_final"
)
.eq(
  "status",
  "Lengkap"
)
```

Selain itu, sistem mengambil daftar dosen yang dapat dipilih sebagai dewan penguji. Kode berikut digunakan untuk menampilkan daftar dosen penguji pada form penjadwalan sidang.

```
const { data: dosenData }
= await supabase
  .from("profiles")
  .select("id, nama")
  .in(
    "role",
    ["dosen", "kaprodi"]
  );
```

Sistem juga memuat data penguji sebelumnya sebagai rekomendasi penguji sidang. Potongan kode berikut digunakan untuk mengisi otomatis rekomendasi dewan penguji berdasarkan data seminar sebelumnya.

```
defaultP1 =
  riwayatPenguji.find(
    p =>
      p.role === "penguji1"
  )?.dosen_id || "";
```

Sistem melakukan validasi agar seluruh data jadwal dan penguji telah terisi sebelum jadwal dikonfirmasi. Kode berikut digunakan untuk memastikan seluruh informasi sidang telah diisi sebelum proses penyimpanan dilakukan.

```
if (
  !tanggal ||
  !jam ||
  !ruangan
)
```

Selain validasi kelengkapan data, sistem juga memastikan dosen penguji tidak boleh sama. Potongan kode berikut digunakan untuk mencegah terjadinya duplikasi dosen penguji pada sidang skripsi.

```
if (
  penguji1 === penguji2 ||
  penguji1 === penguji3
)
```

Sistem kemudian menyimpan jadwal sidang ke tabel *sidang_requests*. Kode tersebut digunakan untuk menyimpan data jadwal sidang mahasiswa ke basis data sistem.

```
await supabase
  .from("sidang_requests")
  .upsert({
    tanggal_sidang:
      tanggal,
    jam_sidang:
      jam
  });
```

Setelah jadwal berhasil dibuat, sistem memperbarui status seminar mahasiswa menjadi siap sidang. Potongan kode berikut digunakan untuk memperbarui tahapan akademik mahasiswa setelah jadwal sidang ditetapkan.

```
await supabase
  .from("seminar_requests")
  .update({
    status:
      "Siap Sidang"
  });
```

Sistem akan mengirimkan notifikasi otomatis kepada mahasiswa dan dosen penguji terkait jadwal sidang yang telah ditetapkan. Kode berikut digunakan untuk memberikan pemberitahuan jadwal sidang kepada seluruh pihak yang terlibat.

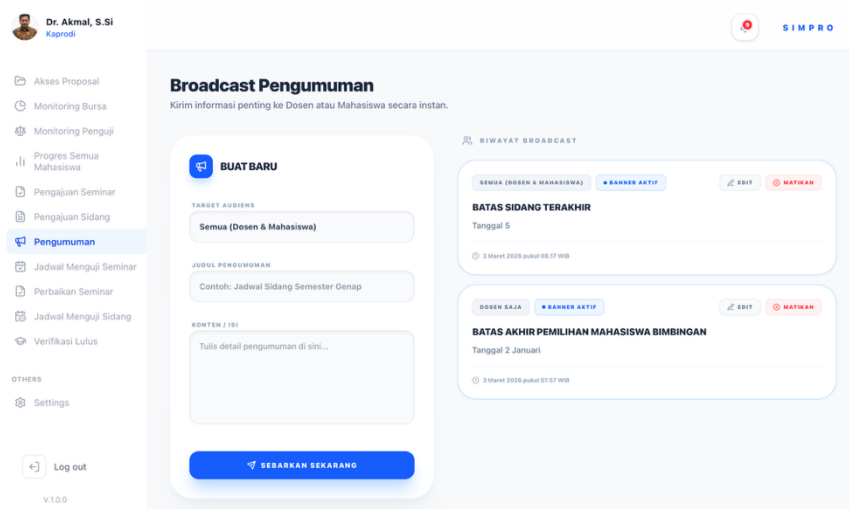
```

await sendNotification(
    studentId,
    "Jadwal Sidang Skripsi",
    message
);

```

4.2.28 Halaman Pengumuman (Kaprodi)

Halaman pengumuman digunakan oleh kaprodi untuk menyebarkan informasi akademik kepada mahasiswa maupun dosen pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO). Pada halaman ini kaprodi dapat membuat pengumuman baru, menentukan target audiens, serta mengelola riwayat *broadcast* pengumuman yang telah dikirimkan.



Gambar 4.28 Halaman pengumuman

Gambar 4.28 menampilkan tampilan halaman pengumuman. Sistem menyediakan form pembuatan pengumuman pada sisi kiri dan riwayat *broadcast* pengumuman pada sisi kanan halaman.

Sistem mengambil data riwayat pengumuman secara *real-time* menggunakan SWR dan Supabase. Potongan kode berikut digunakan untuk mengambil data riwayat pengumuman secara *real-time* dari basis data.

```
const { data: history }
  = useSWR(
    'broadcasts_history_kaprodi',
    fetchBroadcastsHistory
  );
```

Sistem mengambil seluruh data pengumuman dari tabel *broadcasts* dan mengurutkannya berdasarkan waktu terbaru. Kode tersebut digunakan untuk menampilkan daftar riwayat pengumuman yang pernah dibuat oleh kaprodi.

```
const { data }
  = await supabase
    .from('broadcasts')
    .select('*')
    .order(
      'created_at',
      { ascending: false }
    );
```

Sistem menyediakan fitur pembuatan pengumuman baru dengan menentukan target audiens, judul, dan isi pengumuman. Potongan kode berikut digunakan untuk menyimpan data pengumuman baru ke basis data sistem.

```
await supabase
  .from('broadcasts')
  .insert({
    judul,
    konten,
    target_audiens:
      target
  });
```

Selain membuat pengumuman baru, sistem juga menyediakan fitur aktivasi dan penonaktifan *banner* pengumuman. Potongan kode berikut digunakan untuk mengubah status *banner* pengumuman menjadi aktif atau nonaktif.

```
.update({
  is_active:
    !currentStatus
})
```

Sistem juga menyediakan fitur edit pengumuman sehingga kaprodi dapat memperbarui isi pengumuman yang telah dibuat sebelumnya. Kode berikut digunakan untuk memperbarui data pengumuman pada basis data sistem.

```
await supabase
  .from('broadcasts')
  .update({
    judul:
      editingItem.judul,
    konten:
      editingItem.konten
  });
```

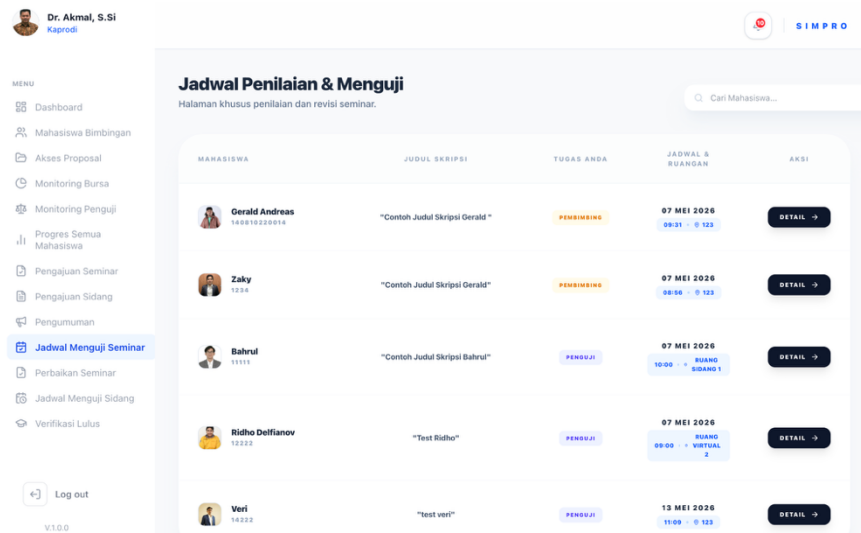
Riwayat pengumuman kemudian ditampilkan dalam bentuk daftar kartu informasi yang memuat target audiens, status *banner*, isi pengumuman, serta waktu publikasi. Potongan kode berikut digunakan untuk menampilkan seluruh riwayat pengumuman secara dinamis pada halaman pengumuman.

```
history.map((item) => (
  <div key={item.id}>
```

4.2.29 Halaman Jadwal Menguji Seminar (Kaprodi & Dosen)

Halaman jadwal menguji seminar digunakan oleh kaprodi dan dosen untuk melihat daftar jadwal seminar mahasiswa yang harus dihadiri. Pada halaman ini kaprodi dan dosen dapat melihat informasi mahasiswa, judul skripsi, perannya dalam seminar, jadwal pelaksanaan seminar, serta mengakses halaman detail penilaian seminar mahasiswa.

Gambar 4.29 menampilkan tampilan halaman jadwal menguji seminar. Sistem menampilkan daftar mahasiswa beserta jadwal seminar yang telah dijadwalkan.



Gambar 4.29 Halaman jadwal menguji seminar

Sistem menggunakan SWR untuk mengambil data jadwal seminar secara *real-time* dari basis data Supabase. Potongan kode berikut digunakan untuk melakukan sinkronisasi data jadwal seminar secara otomatis.

```
const {
  data: schedules = []
} = useSWR(
  'jadwal_menguji_kaprodi_list',
  fetchJadwalMengujiKaprodi
);
```

Sistem mengambil data seminar mahasiswa beserta data jadwal seminar dari basis data. Potongan kode berikut digunakan untuk mengambil informasi jadwal seminar mahasiswa.


```
const { data: requests }
= await supabase
  .from(
    'seminar_requests'
  )
  .select(`
    id,
    proposal_id,
    schedule:
      seminar_schedules (
        tanggal,
        jam,
        ruangan
      )
  `);
```

Sistem juga mengambil data proposal dan identitas mahasiswa melalui relasi tabel proposal dan profil pengguna. Kode berikut digunakan untuk menampilkan identitas mahasiswa dan judul skripsi pada tabel jadwal seminar.

```
proposal:proposals (
  judul,
  user:profiles (
    nama,
    npm,
    avatar_url
  )
)
```

Sistem melakukan *filtering* agar mahasiswa yang telah lulus tidak lagi muncul pada daftar jadwal seminar. Potongan kode berikut digunakan untuk memastikan hanya mahasiswa aktif yang tampil pada halaman jadwal seminar.

```
return (
  prop?.status_lulus
  !== true
);
```

Sistem menentukan peran kaprodi maupun dosen secara otomatis berdasarkan relasi bimbingan mahasiswa. Kode berikut digunakan untuk menentukan apakah dosen bertindak sebagai pembimbing atau penguji seminar.

```

if (
  propIdsFromSupervisor
    .includes(
      reqDetail.proposal_id
    )
)

```

Data hasil *query* kemudian dipetakan ke dalam struktur antarmuka sistem. Potongan kode berikut digunakan untuk membentuk data jadwal seminar yang akan ditampilkan pada tabel.

```

return {
  mahasiswa: {
    nama:
      userData?.nama
  },
  judul_proposal:
    prop?.judul
};

```

4.2.30 Halaman Penilaian Seminar (Kaprodi & Dosen)

Halaman penilaian seminar digunakan oleh kaprodi, dosen pembimbing maupun dosen penguji untuk memberikan penilaian terhadap pelaksanaan seminar mahasiswa. Pada halaman ini kaprodi maupun dosen dapat melihat data mahasiswa, mengunduh draft skripsi, memberikan nilai pada beberapa aspek penilaian, mengunggah file revisi, serta memberikan keputusan revisi seminar mahasiswa.

Gambar 4.30 menampilkan tampilan halaman penilaian seminar. Sistem menyediakan formulir penilaian seminar yang terintegrasi dengan basis data sehingga hasil penilaian dapat tersimpan secara otomatis.

Gambar 4.30 Halaman penilaian seminar

Sistem menggunakan SWR untuk mengambil data seminar secara *real-time* dari basis data. Kode berikut digunakan untuk melakukan pengambilan data seminar dan melakukan pembaruan data secara otomatis.

```
const {
  data: cache
} = useSWR(
  `jadwal_menguji_seminar_${id}`,
  () =>
    fetchDetailUjianSeminar(id)
);
```

Sistem mengambil data seminar mahasiswa, data proposal, data dosen pembimbing, serta dokumen seminar dari basis data Supabase. Potongan kode berikut digunakan untuk mengambil informasi seminar mahasiswa yang akan dinilai oleh dosen.

```
const { data: req }
= await supabase
  .from(
    'seminar_requests'
  )
  .select(`
    id,
    status,
    proposal:
    proposals (
      judul,
      bidang
    )
  `);
```

Sistem juga mengambil data dosen pembimbing dan dosen penguji yang terlibat pada seminar mahasiswa. Dua potongan kode berikut digunakan untuk mengambil data dosen pembimbing dan penguji dari basis data.

```
supabase
  .from(
    'thesis_supervisors'
  )
```

```
supabase
  .from('examiners')
}
```

Sistem menyediakan fitur pengunduhan draf skripsi mahasiswa yang telah diunggah sebelumnya. Kode berikut digunakan untuk membuat tautan sementara agar dosen dapat mengunduh dokumen draf skripsi mahasiswa.

```
createSignedUrl(
  draftDoc.file_url,
  3600
);
```

Sistem melakukan perhitungan otomatis terhadap nilai rata-rata seminar berdasarkan beberapa komponen penilaian. Potongan kode berikut digunakan untuk menghitung nilai rata-rata seminar mahasiswa secara otomatis.

```
const calc =
  (v1 + v2 + v3 +
   v4 + v5) /
  countFilled;
```

Data penilaian seminar kemudian disimpan ke basis data menggunakan proses *upsert*. Kode berikut digunakan untuk menyimpan hasil penilaian seminar dosen ke basis data sistem.

```
await supabase
  .from(
    'seminar_feedbacks'
  )
  .upsert({
    nilai_angka:
      parseFloat(nilai)
  });
```

Sistem juga menyediakan fitur upload dokumen revisi seminar dalam format PDF. Potongan kode berikut digunakan untuk menyimpan file revisi seminar yang diunggah dosen.

```
supabase.storage
  .from(
    'feedback_seminar'
  )
  .upload(
    fileName,
    fileRevisi
  );
```

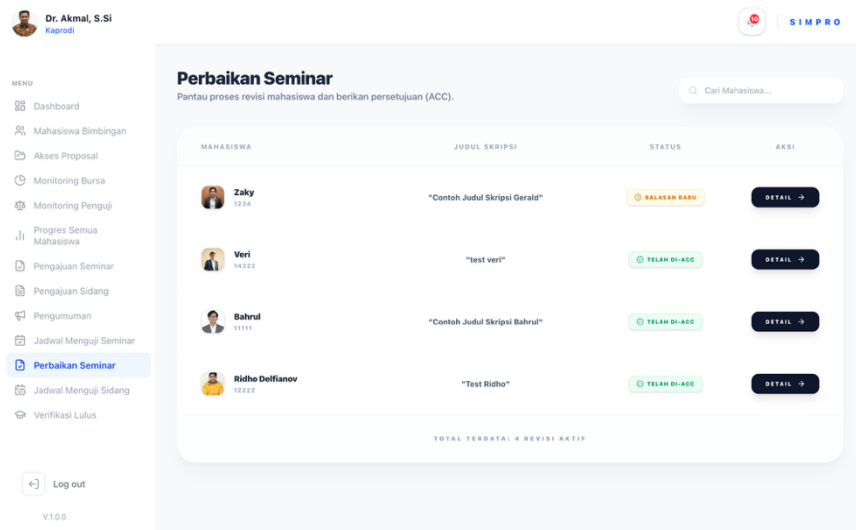
Selain memberikan nilai, dosen juga dapat menentukan status revisi seminar mahasiswa. Kode berikut digunakan untuk memberikan status *ACC* lanjut sidang kepada mahasiswa setelah seminar selesai dilakukan.

```
setStatusRevisi(
  "diterima"
);
```

4.2.31 Halaman Perbaikan Seminar (Kaprodi & Dosen)

Halaman perbaikan seminar digunakan oleh ketua program studi dan dosen untuk memantau proses revisi seminar mahasiswa setelah seminar selesai dilaksanakan. Pada halaman ini kaprodi maupun dosen dapat melihat daftar

mahasiswa yang sedang menjalani proses revisi seminar, status revisi seminar, serta mengakses halaman detail revisi seminar mahasiswa.



Gambar 4.31 Halaman perbaikan seminar

Gambar 4.31 menampilkan tampilan halaman perbaikan seminar. Sistem menampilkan daftar mahasiswa yang sedang melakukan revisi seminar beserta status revisi seminar yang diberikan dosen pembimbing maupun dosen penguji.

Sistem mengambil data revisi seminar mahasiswa berdasarkan dosen yang memberikan revisi. Potongan kode berikut digunakan untuk memperoleh data revisi seminar mahasiswa dari *database*.

```

const { data } =
  await supabase
    .from(
      "seminar_feedbacks"
    )
    .select(`
      id,
      status_revisi,
      seminar_request_id,
      request:seminar_requests (
        proposal:proposals (
          judul,
          status,
          status_lulus,
          user:profiles (
            nama,
            npm,
            avatar_url
          )
        )
      )
    `)
    .eq(
      "dosen_id",
      user.id
    );

```

Sistem melakukan filter data agar mahasiswa yang telah dinyatakan lulus tidak ditampilkan pada daftar revisi seminar aktif. Potongan kode berikut digunakan untuk melakukan penyaringan data mahasiswa.

```

.filter((item) => {
  const prop =
    item.request
      ?.proposal;

  return (
    prop?.status_lulus
      !== true &&
    prop?.status
      !== "Lulus"
  );
})

```

Sistem memformat data revisi seminar mahasiswa sebelum ditampilkan pada tabel daftar revisi seminar. Potongan kode berikut digunakan untuk mempersiapkan data mahasiswa revisi seminar.

```

return {
  request_id:
    item
      .seminar_request_id,
  status:
    item.status_revisi,
  nama:
    prop?.user?.nama,
  npm:
    prop?.user?.npm,
  avatar:
    prop?.user
      ?.avatar_url,
  judul:
    prop?.judul
};

```

Sistem menampilkan status revisi seminar mahasiswa berdasarkan hasil review dosen pembimbing maupun dosen penguji. Potongan kode berikut digunakan untuk menentukan status revisi seminar mahasiswa.

```

const isAcc =
  mhs.status ===
  "diterima";

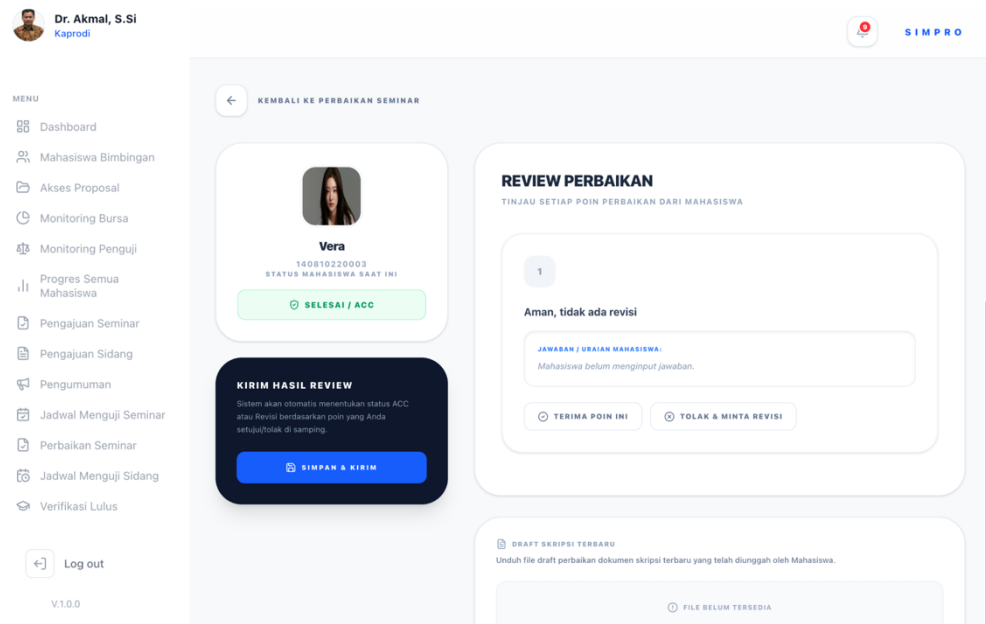
const isReview =
  mhs.status ===
  "diperiksa";

```

4.2.32 Halaman Detail Perbaikan Seminar (Kaprodi & Dosen)

Halaman detail perbaikan seminar digunakan oleh ketua program studi dan dosen untuk meninjau hasil revisi seminar mahasiswa secara lebih rinci. Pada halaman ini kaprodi maupun dosen dapat melihat identitas mahasiswa, daftar poin revisi seminar, jawaban revisi dari mahasiswa, status revisi setiap poin, serta memberikan persetujuan atau penolakan terhadap hasil revisi seminar mahasiswa.

Gambar 4.32 menampilkan tampilan halaman detail perbaikan seminar. Sistem menampilkan daftar poin revisi seminar mahasiswa beserta jawaban revisi yang telah diberikan mahasiswa.



Gambar 4.32 Halaman detail perbaikan seminar

Sistem mengambil data revisi seminar mahasiswa berdasarkan id *seminar_request* yang dipilih. Potongan kode berikut digunakan untuk mengambil data revisi seminar mahasiswa beserta data proposal dan profil mahasiswa.

```
const { data: req } =
  await supabase
    .from(
      "seminar_requests"
    )
    .select(`
      proposal:proposals (
        id,
        judul,
        user:profiles (
          id,
          nama,
          npm,
          avatar_url
        )
      )
    `)
    .eq(
```

```

        "id",
        request_id
    )
    .single();

```

Sistem mengambil data *feedback* revisi seminar yang diberikan dosen pembimbing maupun dosen penguji. Potongan kode berikut digunakan untuk memperoleh data revisi seminar mahasiswa.

```

const { data: fbData } =
    await supabase
        .from(
            "seminar_feedbacks"
        )
        .select("*")
        .eq(
            "seminar_request_id",
            request_id
        )
        .eq(
            "dosen_id",
            user.id
        )
        .single();

```

Sistem mengambil data dosen pembimbing dan dosen penguji yang terlibat pada seminar mahasiswa. Potongan kode berikut digunakan untuk memperoleh data dosen seminar mahasiswa.

```

const {
  data: sups
} = await supabase
  .from(
    "thesis_supervisors"
  )
  .select(`
    role,
    dosen_id,
    dosen:profiles(nama)
  `)
  .eq(
    "proposal_id",
    prop.id
  );
const {
  data: exms
} = await supabase
  .from("examiners")
  .select(`
    role,
    dosen_id,
    dosen:profiles(nama)
  `)
  .eq(
    "seminar_request_id",
    request_id
  );

```

Sistem membaca data poin revisi seminar dan jawaban revisi mahasiswa dalam format JSON sebelum ditampilkan pada halaman review revisi seminar.

Potongan kode berikut digunakan untuk melakukan parsing data revisi seminar.

```

const parsedCatatan =
  JSON.parse(
    feedback
      .catatan_revisi
  );
const parsedJawaban =
  JSON.parse(
    feedback
      .jawaban_revisi
  );

```

Sistem menampilkan daftar poin revisi seminar mahasiswa beserta jawaban revisi yang telah diberikan mahasiswa. Potongan kode berikut digunakan untuk menampilkan data revisi seminar mahasiswa.

```

points =
  parsedCatatan.map(
    (c, i) => ({
      ...c,
      student_answer:
        parsedJawaban[i]
          ?.answerText || ""
    })
  );

```

Sistem menyediakan fitur untuk menerima atau menolak setiap poin revisi seminar mahasiswa. Potongan kode berikut digunakan untuk menentukan status revisi seminar pada setiap poin revisi.

```

const handlePointStatus =
  (
    idx,
    status
  ) => {
    const newPoints =
      [...pointData];
    newPoints[idx]
      .status =
        status;
    setPointData(
      newPoints
    );
  };

```

Sistem menyimpan alasan penolakan revisi seminar apabila terdapat poin revisi yang belum sesuai. Potongan kode berikut digunakan untuk menyimpan alasan penolakan revisi seminar mahasiswa.

```

const handlePointFeedback =
  (
    idx,
    text
  ) => {
    const newPoints =
      [...pointData];
    newPoints[idx]
      .dosen_feedback =
        text;
    setPointData(
      newPoints
    );
  };

```

Sistem menentukan status akhir revisi seminar mahasiswa berdasarkan hasil *review* setiap poin revisi seminar. Potongan kode berikut digunakan untuk menentukan status revisi seminar mahasiswa.

```
const isAnyRejected =
  pointData.some(
    p =>
      p.status ===
        "rejected"
  );
const isAllApproved =
  pointData.every(
    p =>
      p.status ===
        "approved"
  );
```

Sistem menyimpan hasil *review* revisi seminar mahasiswa ke *database*. Potongan kode berikut digunakan untuk memperbarui status revisi seminar mahasiswa.

```
await supabase
  .from(
    "seminar_feedbacks"
  )
  .update({
    catatan_revisi:
      JSON.stringify(
        newCatatan
      ),
    status_revisi:
      newGlobalStatus
  })
  .eq(
    "id",
    feedback.id
  );
```

Sistem mengirimkan notifikasi otomatis kepada mahasiswa setelah hasil *review* revisi seminar disimpan. Potongan kode berikut digunakan untuk mengirim notifikasi kepada mahasiswa.

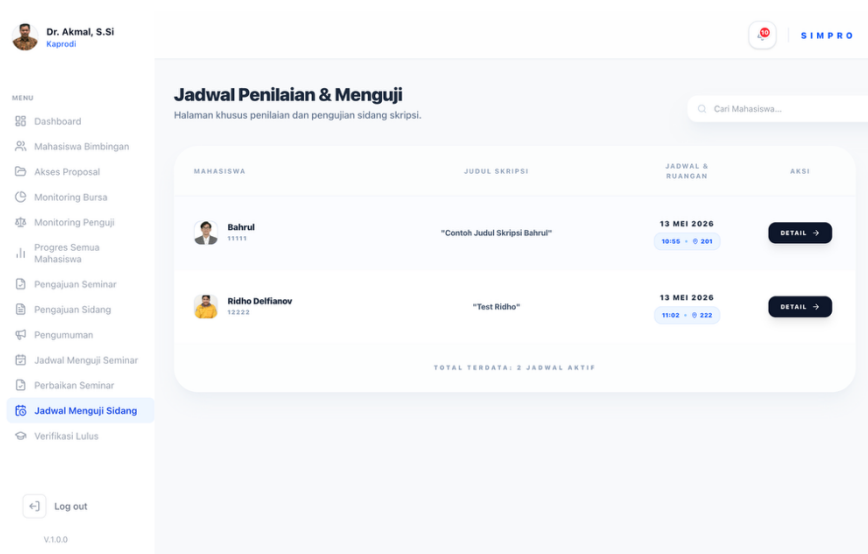
```
await sendNotification(
  studentId,
  titleNotif,
  messageNotif
);
```

Sistem menyediakan fitur untuk mengunduh draf revisi skripsi terbaru yang telah diunggah mahasiswa. Potongan kode berikut digunakan untuk menghasilkan URL sementara file revisi mahasiswa.

```
const { data: mhsFile } =
  await supabase.storage
    .from(
      "seminar_perbaikan"
    )
    .createSignedUrl(
      fbData
        .file_jawaban_url,
      3600
    );
```

4.2.33 Halaman Jadwal Menguji Sidang (Kaprodi & Dosen)

Halaman jadwal menguji sidang digunakan oleh ketua program studi dan dosen untuk melihat daftar jadwal sidang mahasiswa yang harus dinilai dan diuji. Pada halaman ini kaprodi maupun dosen dapat melihat informasi mahasiswa, judul skripsi, jadwal pelaksanaan sidang, ruangan sidang, serta mengakses halaman penilaian sidang mahasiswa.



Gambar 4.33 Halaman Jadwal menguji sidang

Gambar 4.33 menampilkan tampilan halaman jadwal menguji sidang. Sistem menampilkan daftar mahasiswa beserta jadwal sidang yang telah dijadwalkan.

Sistem mengambil data jadwal sidang mahasiswa berdasarkan dosen penguji yang terlibat pada sidang mahasiswa. Potongan kode berikut digunakan untuk memperoleh data jadwal sidang mahasiswa.

```

const {
  data: sidangRequests
} = await supabase
  .from("sidang_requests")
  .select(`
    id,
    proposal_id,
    tanggal_sidang,
    jam_sidang,
    ruangan,
    penguji1,
    penguji2,
    penguji3,
    status,
    proposal:proposals (
      judul,
      status,
      status_lulus,
      user:profiles (
        nama,
        npm,
        avatar_url
      )
    )
  `);

```

Sistem hanya menampilkan jadwal sidang mahasiswa yang masih aktif dan belum dinyatakan lulus. Potongan kode berikut digunakan untuk melakukan penyaringan data jadwal sidang mahasiswa.

```

const myRequests =
  (
    sidangRequests || []
  ).filter(
    (req) => {
      const prop =
        Array.isArray(
          req.proposal
        )
        ? req.proposal[0]
        : req.proposal;

      const isLulus =
        prop
          ?.status_lulus ===
          true ||
        prop?.status ===
          "Lulus";

      return !isLulus;
    }
  );

```


Sistem mengurutkan jadwal sidang mahasiswa berdasarkan tanggal sidang terdekat. Potongan kode berikut digunakan untuk mengurutkan data jadwal sidang mahasiswa.

```
const finalData =
  mappedData.sort(
    (a, b) =>
      new Date(
        a.tanggal
      ).getTime() -
      new Date(
        b.tanggal
      ).getTime()
  );
```

4.2.34 Halaman Penilaian Sidang (Kaprodi & Dosen)

Halaman penilaian sidang digunakan oleh ketua program studi dan dosen untuk memberikan penilaian terhadap sidang skripsi mahasiswa. Pada halaman ini pengguna dapat melihat identitas mahasiswa, judul skripsi, dokumen skripsi *final*, serta mengisi komponen penilaian sidang mahasiswa.

Gambar 4.34 Halaman penilaian sidang

Gambar 4.34 menampilkan tampilan halaman penilaian sidang. Sistem menampilkan formulir penilaian sidang yang terdiri dari beberapa komponen penilaian dan hasil rata-rata nilai akhir mahasiswa.

Sistem mengambil data sidang mahasiswa berdasarkan id sidang yang dipilih. Potongan kode berikut digunakan untuk memperoleh data sidang mahasiswa beserta informasi proposal dan profil mahasiswa.

```
const {
  data: sidang
} = await supabase
  .from("sidang_requests")
  .select(`
    id,
    status,
    proposal:proposals (
      id,
      judul,
      bidang,
      user:profiles (
        nama,
        npm,
        avatar_url
      )
    )
  `)
  .or(
    `id.eq.${id},
    seminar_request_id.eq.${id}`
  )
  .maybeSingle();
```

Sistem mengambil file skripsi *final* mahasiswa yang telah diunggah sebelumnya. Potongan kode berikut digunakan untuk memperoleh dokumen skripsi *final* mahasiswa.

```
const {
  data: docSidang
} = await supabase
  .from(
    "sidang_documents_verification"
  )
  .select("file_url")
  .eq(
    "proposal_id",
    propData.id
  )
  .eq(
    "nama_dokumen",
    "file_skripsi_final"
  )
  .maybeSingle();
```

Sistem menghasilkan URL sementara agar dokumen skripsi dapat diakses secara aman oleh kaprodi. Potongan kode berikut digunakan untuk membuat *signed* URL dokumen skripsi.

```
const {
  data: signedData
} = await supabase.storage
  .from("docseminar")
  .createSignedUrl(
    docSidang.file_url,
    3600
  );
draftUrl =
  signedData
    ?.signedUrl || null;
```

Sistem menghitung rata-rata nilai sidang secara otomatis berdasarkan komponen penilaian yang diinputkan. Potongan kode berikut digunakan untuk melakukan perhitungan nilai akhir mahasiswa.

```

const calc =
(
  v1 +
  v2 +
  v3 +
  v4 +
  v5
) / countFilled;
setNilai(
  calc.toFixed(2)
);

```

Sistem melakukan konversi nilai akhir menjadi huruf mutu secara otomatis.

Potongan kode berikut digunakan untuk menentukan huruf mutu mahasiswa.

```

if (calc >= 80)
  setHurufMutu("A");
else if (calc >= 70)
  setHurufMutu("B");
else if (calc >= 60)
  setHurufMutu("C");
else if (calc >= 50)
  setHurufMutu("D");
else
  setHurufMutu("E");

```

Sistem menyimpan data nilai sidang mahasiswa ke *database* setelah dosen menekan tombol simpan nilai. Potongan kode berikut digunakan untuk menyimpan nilai sidang mahasiswa.

```

await supabase
  .from("sidang_grades")
  .upsert(
    {
      sidang_request_id:
        data.id,
      dosen_id:
        user.id,
      nilai_akhir:
        parseFloat(nilai),
      detail_nilai:
        detailNilaiJson,
    },
    {
      onConflict:
        "sidang_request_id,dosen_id"
    }
  );

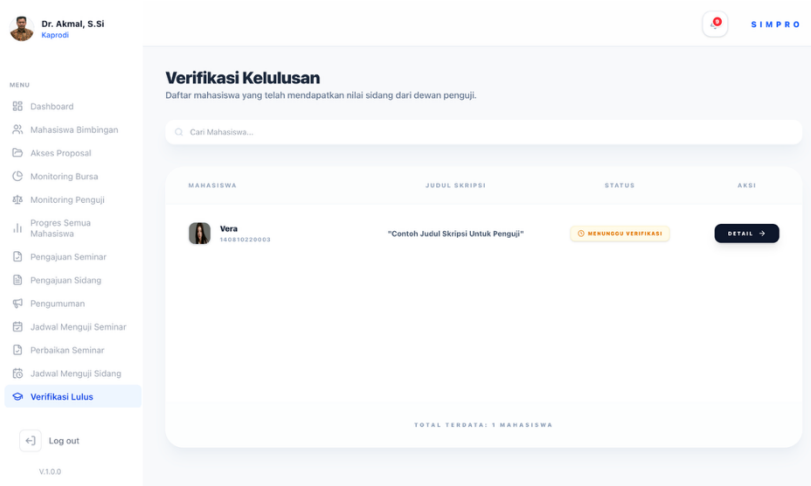
```

Sistem mengunci formulir penilaian apabila kaprodi maupun dosen telah mengirimkan nilai sidang mahasiswa. Potongan kode berikut digunakan untuk mengatur status penguncian penilaian.

```
setIsLocked(true);
```

4.2.35 Halaman Verifikasi Kelulusan

Halaman verifikasi kelulusan digunakan oleh ketua program studi untuk memantau mahasiswa yang telah menyelesaikan sidang skripsi dan memperoleh penilaian dari dewan penguji. Pada halaman ini kaprodi dapat melihat daftar mahasiswa, judul skripsi, status kelulusan, serta mengakses halaman detail verifikasi nilai sidang mahasiswa.



Gambar 4.35 Halaman verifikasi kelulusan

Gambar 4.35 menampilkan tampilan halaman verifikasi kelulusan. Sistem menampilkan daftar mahasiswa yang telah memperoleh nilai sidang dan menunggu proses verifikasi kelulusan oleh kaprodi.

Sistem menggunakan SWR untuk mengambil data verifikasi kelulusan secara realtime dari basis data Supabase. Potongan kode berikut digunakan untuk melakukan sinkronisasi data verifikasi kelulusan secara otomatis tanpa perlu memuat ulang halaman.

```
const {
  data: listMahasiswa = []
} = useSWR(
  'verifikasi_kelulusan_kaprodi_list',
  fetchDaftarMahasiswaSidang
);
```

Sistem mengambil data sidang mahasiswa beserta data proposal dan profil mahasiswa dari basis data. Kode berikut digunakan untuk memperoleh data mahasiswa yang telah melaksanakan sidang skripsi.

```
const { data: sidangData }
= await supabase
  .from("sidang_requests")
  .select(`
    proposal:proposals (
      judul,
      status_lulus,
      user:profiles (
        nama,
        npm
      )
    )
  `);
```

Sistem juga melakukan *filtering* agar hanya mahasiswa yang belum diverifikasi kelulusannya yang ditampilkan pada halaman. Potongan kode berikut digunakan untuk memastikan mahasiswa yang telah dinyatakan lulus tidak muncul kembali pada daftar verifikasi.

```
.eq(
  'proposal.status_lulus',
  false
);
```

Sistem menghitung jumlah dosen penguji yang telah memberikan nilai sidang mahasiswa. Kode berikut digunakan untuk memastikan hanya mahasiswa yang telah memperoleh penilaian sidang yang dapat diverifikasi kelulusannya oleh kaprodi.

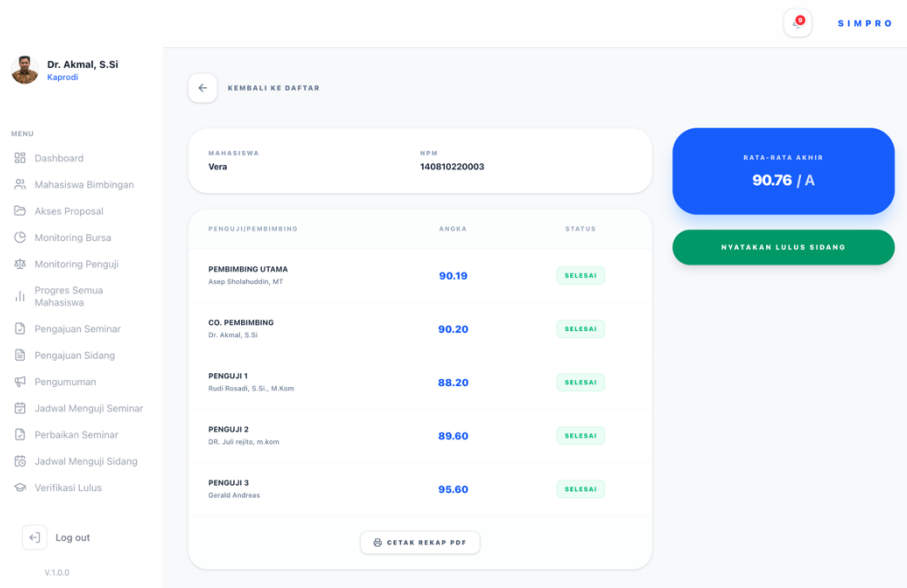
```
const countPenilai =
  gradesList.filter(
    (g) =>
      g.sidang_request_id
      === item.id
  ).length;
```

Sistem menyediakan tombol detail untuk membuka halaman detail verifikasi nilai sidang mahasiswa. Kode berikut digunakan untuk mengarahkan kaprodi menuju halaman detail verifikasi kelulusan mahasiswa.

```
router.push(
  `/kaprodi/
  detailverifikasinilai`
);
```

4.2.36 Halaman Detail Verifikasi Kelulusan

Halaman detail verifikasi kelulusan digunakan oleh ketua program studi untuk melihat rekapitulasi nilai sidang mahasiswa sebelum menetapkan status kelulusan. Pada halaman ini kaprodi dapat melihat identitas mahasiswa, daftar nilai dari dosen pembimbing dan penguji, rata-rata nilai akhir, mencetak rekap berita acara sidang, serta memberikan keputusan kelulusan mahasiswa.



Gambar 4.36 Halaman detail verifikasi kelulusan

Gambar 4.36 menampilkan tampilan halaman detail verifikasi kelulusan. Sistem menampilkan rekap nilai sidang mahasiswa beserta tombol verifikasi kelulusan yang hanya dapat digunakan apabila seluruh penilaian dosen telah sepenuhnya lengkap.

Sistem menggunakan parameter id dari URL untuk mengambil data sidang mahasiswa yang dipilih. Potongan kode berikut digunakan untuk menentukan data sidang mahasiswa yang akan diverifikasi oleh kaprodi.

```
import NewsList from "@/components/news/NewsList";
const sidangId =
  searchParams.get(
    'id'
  );
```


Sistem menggunakan SWR untuk mengambil data detail nilai sidang secara *real-time*. Kode berikut digunakan untuk melakukan sinkronisasi data penilaian sidang secara otomatis tanpa perlu memuat ulang halaman.

```
const {
  data: cache
} = useSWR(
  sidangId
  ? `verifikasi_nilai_${sidangId}`
  : null,
  () =>
    fetchDetailNilaiSidang(
      sidangId
    )
);
```

Sistem mengambil data sidang mahasiswa beserta data proposal dan profil mahasiswa dari basis data Supabase. Potongan kode berikut digunakan untuk memperoleh identitas mahasiswa dan informasi sidang skripsi.

```
const { data: sReq }
= await supabase
  .from(
    'sidang_requests'
  )
  .select(`
    proposal:proposals (
      judul,
      status_lulus,
      user:profiles (
        nama,
        npm
      )
    )
  `);
```

Sistem juga mengambil data dosen pembimbing dan dosen penguji yang terlibat pada seminar maupun sidang mahasiswa. Kode berikut digunakan untuk memperoleh data dosen pembimbing dan dosen penguji mahasiswa.

```

supabase
  .from(
    'thesis_supervisors'
  );
supabase
  .from(
    'examiners'
  );

```

Sistem mengambil data nilai seminar dan nilai sidang dari basis data.

Potongan kode berikut digunakan untuk memperoleh data penilaian dari seluruh dosen penguji.

```

const { data: sidGrades }
= await supabase
  .from(
    'sidang_grades'
  )
  .select(
    'dosen_id,
    nilai_akhir'
  );

```

Sistem melakukan perhitungan otomatis terhadap rata-rata nilai akhir mahasiswa. Kode berikut digunakan untuk menghitung rata-rata nilai sidang mahasiswa berdasarkan seluruh penilaian dosen pembimbing dan penguji.

```

nilaiAkhir =
  totalNilai / 5;

```

Sistem juga menyediakan fitur pencetakan berita acara sidang dalam format PDF. Kode berikut digunakan untuk membuka halaman cetak rekapitulasi nilai sidang mahasiswa.

```

window.open(
  '',
  '_blank'
);

```

Sistem melakukan validasi bahwa seluruh dosen telah memberikan nilai sebelum mahasiswa dapat dinyatakan lulus. Potongan kode berikut digunakan

untuk memastikan seluruh komponen penilaian telah lengkap sebelum proses verifikasi kelulusan dilakukan.

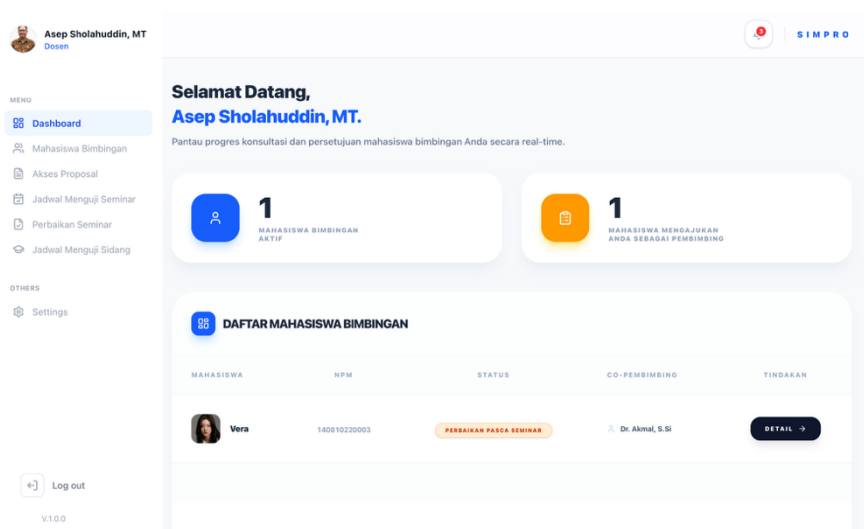
```
const isComplete =
  countNilai === 5;
```

Setelah seluruh nilai lengkap, kaprodi dapat menetapkan status kelulusan mahasiswa. Kode berikut digunakan untuk memperbarui status kelulusan mahasiswa pada basis data sistem.

```
await supabase
  .from('proposals')
  .update({
    status_lulus: true
  });
```

4.2.37 Halaman *Dashboard* Dosen

Halaman *dashboard* dosen digunakan oleh dosen untuk memantau aktivitas bimbingan mahasiswa secara *real-time*. Pada halaman ini dosen dapat melihat jumlah mahasiswa bimbingan aktif, jumlah mahasiswa yang mengajukan dosen sebagai pembimbing, informasi pengumuman akademik, serta daftar mahasiswa bimbingan beserta status progres skripsinya.



Gambar 4.37 Halaman *dashboard* dosen

Gambar 4.37 menampilkan tampilan halaman *dashboard* dosen. Sistem menampilkan ringkasan aktivitas bimbingan dan daftar mahasiswa yang sedang berada dalam proses pengerjaan skripsi.

Sistem menggunakan SWR untuk mengambil data *dashboard* dosen secara *real-time*. Potongan kode berikut digunakan untuk melakukan sinkronisasi data *dashboard* dosen secara otomatis tanpa perlu memuat ulang halaman.

```
const { data, isLoading }
= useSWR(
  'dashboard_dosen_data',
  fetchDashboardDosenData
);
```

Sistem mengambil data akun dosen yang sedang *login* menggunakan autentikasi Supabase. Kode berikut digunakan untuk memperoleh identitas dosen yang sedang mengakses sistem.

```
const {
  data: { user }
} = await supabase
  .auth
  .getUser();
```

Sistem menjalankan beberapa *query* secara paralel untuk meningkatkan performa pengambilan data *dashboard*. Potongan kode berikut digunakan agar proses pengambilan data *dashboard* menjadi lebih cepat dan efisien.

```
const [
  profileRes,
  bannerRes,
  propDataRes,
  bimbinganRes
] = await Promise.all([
  ...
]);
```

Sistem menghitung jumlah mahasiswa yang mengajukan dosen yang bersangkutan sebagai pembimbing. Kode berikut digunakan untuk menentukan jumlah pengajuan mahasiswa yang belum direspons oleh dosen.

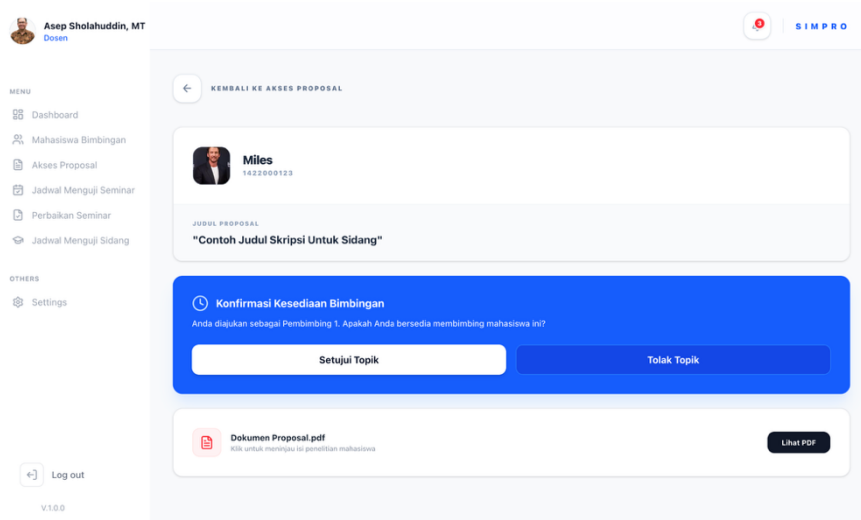
```
if (
  isRecommended &&
  !hasResponded
) {
  pendingRequestsCount++;
}
```

Sistem mengambil data mahasiswa bimbingan beserta informasi seminar, sidang, dokumen, dan sesi bimbingan mahasiswa. Potongan kode berikut digunakan untuk memperoleh data progres skripsi mahasiswa secara lengkap.

```
supabase
  .from(
    "thesis_supervisors"
  )
  .select(`
    proposal_id,
    proposal:proposals (
      seminar:seminar_requests,
      sidang:sidang_requests,
      docs:seminar_documents,
      sessions:guidance_sessions
    )
  `);
```

4.2.38 Halaman Detail Proposal (Dosen)

Halaman detail proposal digunakan oleh dosen untuk melihat detail usulan proposal mahasiswa serta memberikan respon terhadap permintaan pembimbingan. Pada halaman ini dosen dapat melihat identitas mahasiswa, judul proposal, dokumen proposal skripsi, serta memberikan keputusan menerima atau menolak pengajuan pembimbingan mahasiswa.



Gambar 4.38 Halaman detail proposal dosen

Gambar 4.38 menampilkan tampilan halaman detail proposal dosen. Sistem menampilkan informasi proposal mahasiswa beserta fitur konfirmasi kesediaan dosen sebagai pembimbing.

Sistem mengambil data proposal mahasiswa berdasarkan id proposal yang dipilih. Potongan kode berikut digunakan untuk mengambil informasi proposal mahasiswa beserta data profil mahasiswa.

```
const { data: propData } =
  await supabase
    .from("proposals")
    .select(`
      id,
      judul,
      file_path,
      status,
      user:profiles (
        id,
        nama,
        npm
      )
    `)
    .eq("id", proposalId)
    .maybeSingle();
```

Sistem mengambil data dosen pembimbing yang telah memberikan respon terhadap proposal mahasiswa. Kode berikut digunakan untuk memperoleh data status persetujuan atau penolakan dosen pembimbing.

```
const {
  data: supervisors
} = await supabase
  .from(
    "thesis_supervisors"
  )
  .select(`
    id,
    dosen_id,
    role,
    status
  `)
  .eq(
    "proposal_id",
    proposalId
  );
```

Sistem mengambil data rekomendasi dosen pembimbing yang dipilih mahasiswa. Potongan kode berikut digunakan untuk menampilkan dosen yang direkomendasikan sebagai pembimbing proposal mahasiswa.

```
const {
  data: recommendations
} = await supabase
  .from(
    "proposal_recommendations"
  )
  .select(`
    dosen_id,
    tipe
  `)
  .eq(
    "proposal_id",
    proposalId
  );
```

Sistem memungkinkan dosen menerima pengajuan bimbingan mahasiswa. Potongan kode berikut digunakan untuk menyimpan status persetujuan atau penolakan dosen pembimbing ke *database*.

```
status:
  isAccepted
    ? "accepted"
    : "rejected"
```

Apabila dosen menolak usulan pembimbingan, sistem akan menyimpan alasan penolakan dan hasil penilaian terhadap kelayakan proposal. Potongan kode berikut digunakan untuk menyimpan alasan penolakan dan status kelayakan proposal ke dalam basis data.

```
rejection_reason:
isAccepted
?null
: rejectReason

is_layak:
isAccepted
?null
: isLayak
```

Apabila dosen menolak pengajuan dan menilai proposal tidak layak, sistem akan mengubah status proposal menjadi ditolak dosbing dan membuka kembali akses unggah dokumen sehingga mahasiswa dapat melakukan revisi proposal. Potongan kode berikut digunakan untuk memperbarui status proposal dan membuka kembali akses pengajuan mahasiswa.

```
await supabase
  .from("proposals")
  .update({
    status:
      "Ditolak Dosbing",
    is_locked: false
  })
  .eq("id", proposalId);
```


Sebaliknya, apabila dosen menolak pengajuan tetapi menilai proposal masih layak, mahasiswa tidak diwajibkan melakukan revisi. Dalam kondisi tersebut kaprodi akan mencari dosen pembimbing lain yang sesuai dengan bidang kajian proposal.

Sistem mengirimkan notifikasi otomatis kepada kaprodi setelah dosen memberikan respon. Potongan kode berikut digunakan untuk memberikan pemberitahuan kepada kaprodi terkait hasil respon dosen pembimbing.

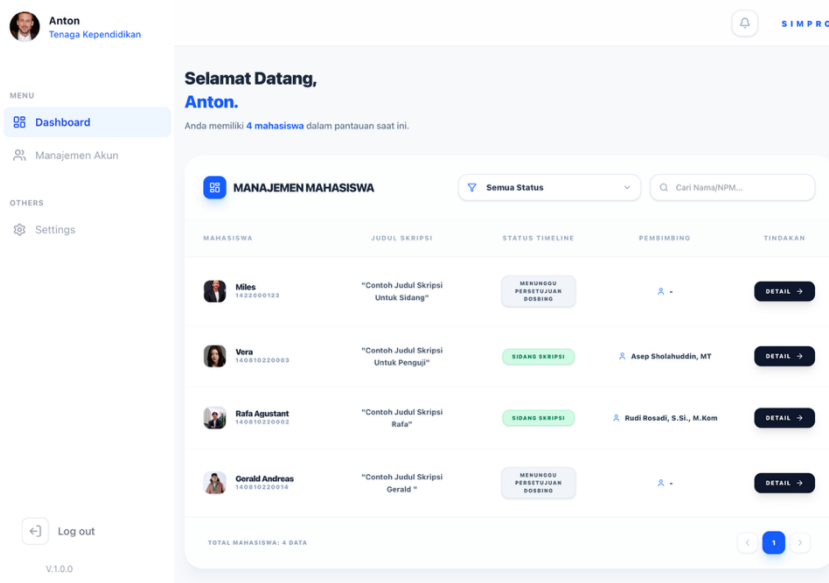
```
await supabase
  .from(
    "notifications"
  )
  .insert(
    notifications
  );
```

Sistem menyediakan fitur untuk membuka dokumen proposal mahasiswa secara langsung. Kode berikut digunakan untuk menghasilkan URL sementara agar dokumen proposal dapat diakses dosen secara aman.

```
const { data } =
  await supabase.storage
    .from("proposals")
    .createSignedUrl(
      proposal.file_path,
      3600
    );
```

4.2.39 Halaman *Dashboard* Tenaga Kependidikan

Halaman *dashboard* tenaga kependidikan digunakan oleh tendik untuk memantau progres skripsi mahasiswa. Pada halaman ini tendik dapat melihat daftar mahasiswa, status progres skripsi, dosen pembimbing, serta melakukan pencarian dan filter data mahasiswa berdasarkan tahapan skripsi.



Gambar 4.39 Halaman dashboard tendik

Gambar 4.39 menampilkan tampilan halaman *dashboard* tendik. Sistem menampilkan daftar mahasiswa beserta status perkembangan skripsi mahasiswa secara *real-time*.

Sistem menggunakan metode SWR untuk memperbarui data *dashboard* secara otomatis tanpa perlu memuat ulang halaman. Potongan kode berikut digunakan untuk melakukan *refresh* data secara berkala.

```
const {
  data: fetchResult,
  isLoading
} = useSWR(
  'dashboard_tendik_main',
  fetchDashboardTendikData,
  {
    revalidateOnFocus:
      true,
    refreshInterval:
      60000
  }
);
```

Sistem mengambil data proposal mahasiswa beserta informasi pembimbing, seminar, sidang, dan bimbingan mahasiswa dari *database*. Potongan kode berikut digunakan untuk mengambil data monitoring mahasiswa.

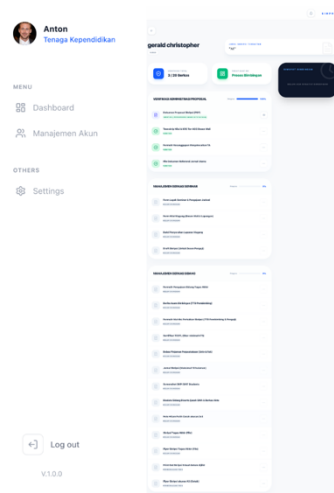
```
const {
  data: proposals,
  error
} = await supabase
  .from("proposals")
  .select(`
    id,
    judul,
    status,
    status_lulus,
    profiles (
      nama,
      npm,
      avatar_url
    ),
    thesis_supervisors (
      role,
      profiles ( nama )
    ),
    seminar_requests (
      id,
      status
    ),
    sidang_requests (
      id,
      status
    )
  `)
  .eq(
    'status_lulus',
    false
  );
```

Sistem melakukan pemetaan status progres mahasiswa berdasarkan tahapan skripsi mahasiswa. Potongan kode berikut digunakan untuk menentukan status *timeline* mahasiswa.

```
const ui =
  mapStatusToUI ({
    proposalStatus:
      p.status,
    hasSeminar:
      !!activeSeminarReq,
    seminarStatus:
      activeSeminarReq?.status,
    hasSidang:
      hasSidang
  });
```

4.2.40 Halaman Detail Progres (Tendik)

Halaman detail progres digunakan oleh tenaga kependidikan untuk memantau detail progres mahasiswa skripsi serta melakukan verifikasi berkas seminar dan sidang. Pada halaman ini tendik dapat melihat identitas mahasiswa, status tahapan skripsi, progres kelengkapan berkas, riwayat bimbingan, serta melakukan validasi dokumen mahasiswa.



Gambar 4.40 Halaman detail progres tendik

Gambar 4.40 menampilkan tampilan halaman detail progres. Sistem menampilkan informasi mahasiswa beserta daftar dokumen seminar, dokumen sidang, dan riwayat bimbingan mahasiswa.

Sistem menggunakan metode SWR untuk memperbarui data progres mahasiswa secara otomatis tanpa perlu memuat ulang halaman. Potongan kode berikut digunakan untuk melakukan *refresh* data secara berkala.

```
const {
  data: cache,
  isLoading,
  mutate
} = useSWR(
  proposalId
  ? `detail_progres_${proposalId}`
  : null,
  () =>
    fetchDetailTendikData(
      proposalId
    ),
  {
    revalidateOnFocus:
      true,
    refreshInterval:
      60000
  }
);
```

Sistem menghitung jumlah dokumen yang telah diverifikasi untuk menentukan progres kelengkapan berkas mahasiswa. Potongan kode berikut digunakan untuk menghitung jumlah dokumen valid.

```
const verifiedDocsCount =
  currentDocs.filter(
    d => d.status ===
      'Lengkap'
  ).length;
```

Sistem menentukan tahapan progres mahasiswa berdasarkan status seminar, sidang, dan hasil bimbingan mahasiswa. Potongan kode berikut digunakan untuk memetakan status progres mahasiswa.

```
const ui =
  mapStatusToUI({
    proposalStatus:
      propData.status,
    hasSeminar:
      !!activeSeminarReq,
    seminarStatus:
      activeSeminarReq?.status,
    hasSidang:
      !!sidangData,
    uploadedDocsCount:
      currentDocs.length,
    verifiedDocsCount:
      verifiedDocsCount
  });
```

Sistem mengambil data riwayat bimbingan mahasiswa yang telah disetujui dosen pembimbing. Potongan kode berikut digunakan untuk memperoleh riwayat sesi bimbingan mahasiswa.

```
const {
  data: bimData
} = await supabase
  .from(
    "guidance_sessions"
  )
  .select(`
    id,
    sesi_ke,
    tanggal,
    dosen:profiles (
      nama
    )
  `)
  .eq(
    "proposal_id",
    proposalId
  )
  .eq(
    "kehadiran_mahasiswa",
    "hadir"
  );
```

Sistem menyediakan fitur verifikasi dokumen administrasi proposal, seminar, dan sidang mahasiswa. Untuk dokumen administrasi proposal, status verifikasi disimpan langsung pada data proposal. Potongan kode berikut digunakan untuk memperbarui status dokumen yang telah diverifikasi oleh tendik.

```

await supabase
  .from("proposals")
  .update({
    [docObj.statusKey]:
    newStatus
  })
  .eq(
    "id",
    proposalId
  );

```

Apabila terdapat dokumen yang ditolak, sistem akan mengubah status proposal menjadi ditolak tendik dan membuka kembali akses pengajuan agar mahasiswa dapat mengunggah dokumen perbaikan. Potongan kode berikut digunakan untuk memperbarui status proposal setelah dokumen ditolak.

```

await supabase
  .from("proposals")
  .update({
    status:
    "Ditolak Tendik",
    is_locked: false
  });

```

Sistem mengirimkan notifikasi otomatis kepada mahasiswa setelah dokumen diverifikasi atau ditolak oleh tendik. Potongan kode berikut digunakan untuk mengirim notifikasi kepada mahasiswa.

```

await sendNotification(
  student.user.id,
  title,
  message
);

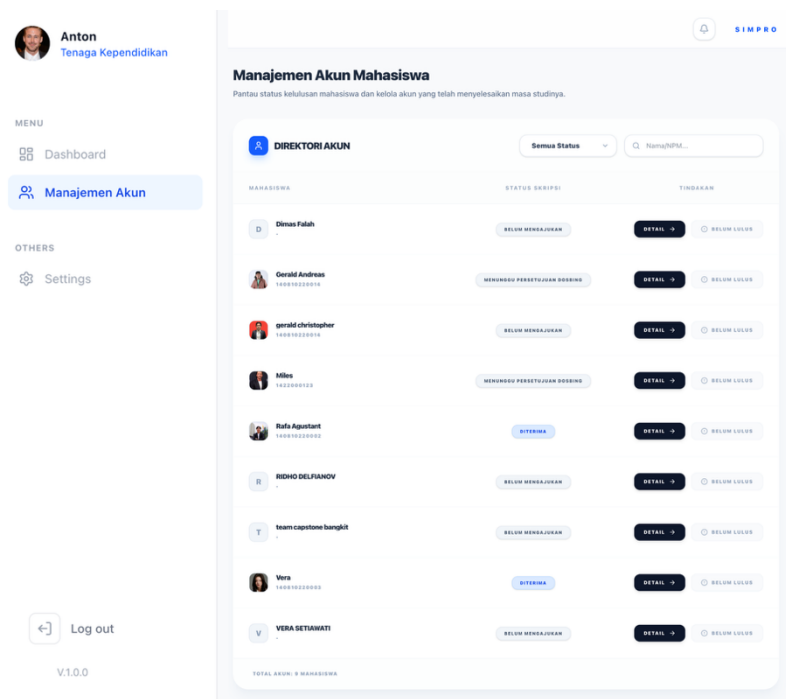
```

Sistem menyediakan fitur untuk membuka dan mengunduh dokumen mahasiswa secara aman menggunakan *signed* URL dari Supabase *Storage*. Potongan kode berikut digunakan untuk menghasilkan URL dokumen sementara.

```
const {
  data
} = await supabase
  .storage
  .from("docseminar")
  .createSignedUrl(
    normalizeStoragePath(
      rawPath
    ),
    3600
  );
```

4.2.41 Halaman Manajemen Akun (Tendik)

Halaman manajemen akun digunakan oleh tenaga kependidikan untuk memantau dan mengelola akun mahasiswa pada sistem. Pada halaman ini tendik dapat melihat daftar akun mahasiswa, status skripsi mahasiswa, melakukan pencarian data mahasiswa, memfilter status skripsi, melihat detail mahasiswa, serta menghapus akun mahasiswa yang telah lulus.



Gambar 4.41 Halaman manajemen akun

Gambar 4.41 menampilkan tampilan halaman manajemen akun. Sistem menampilkan daftar akun mahasiswa beserta status skripsi dan fitur pengelolaan akun mahasiswa.

Sistem menggunakan metode SWR untuk memperbarui data akun mahasiswa secara otomatis tanpa perlu memuat ulang halaman. Potongan kode berikut digunakan untuk melakukan *refresh* data akun mahasiswa secara berkala.

```
const {
  data: students,
  error,
  isLoading,
  mutate
} = useSWR(
  "tendik_manajemen_akun",
  fetchMahasiswa,
  {
    revalidateOnFocus:
      true,
    refreshInterval:
      60000
  }
);
```

Sistem mengambil data akun mahasiswa beserta status proposal skripsi mahasiswa dari *database*. Potongan kode berikut digunakan untuk memperoleh data mahasiswa dan status skripsi mahasiswa.

```
const {
  data,
  error
} = await supabase
  .from("profiles")
  .select(`
    id,
    nama,
    npm,
    email,
    avatar_url,
    proposal:proposals (
      status,
      status_lulus
    )
  `)
  .eq(
    "role",
```

```

    "mahasiswa"
  )
  .order(
    "nama",
    { ascending: true }
  );

```

Sistem menentukan status akhir skripsi mahasiswa berdasarkan status proposal dan status kelulusan mahasiswa. Potongan kode berikut digunakan untuk menentukan status skripsi mahasiswa.

```

let finalStatus =
  "Belum Mengajukan";
if (prop) {
  if (
    prop.status_lulus ===
    true ||
    prop.status ===
    "Lulus"
  ) {
    finalStatus =
      "Lulus";
  } else {
    finalStatus =
      prop.status ||
      "Belum Mengajukan";
  }
}

```

Sistem memungkinkan tendik menghapus akun mahasiswa yang telah lulus dari sistem. Potongan kode berikut digunakan untuk menghapus akun mahasiswa melalui *API*.

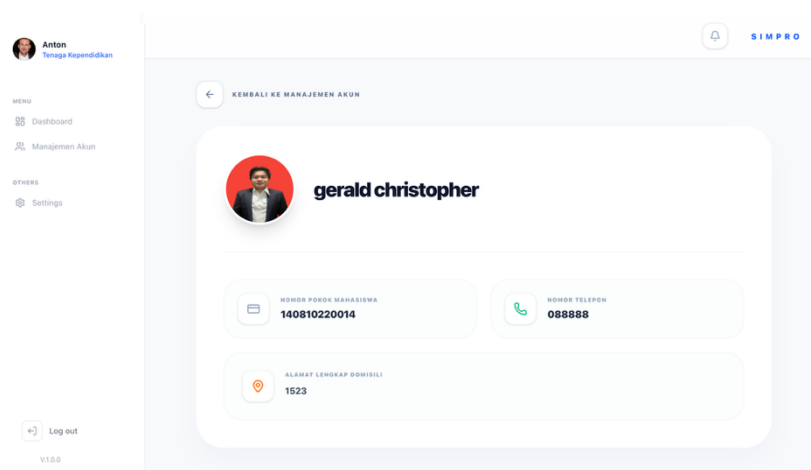
```

const res =
  await fetch(
    "/api/delete-user",
    {
      method: "DELETE",
      headers: {
        "Content-Type":
          "application/json"
      },
      body: JSON.stringify({
        userId
      })
    }
  );

```

4.2.42 Halaman Detail Manajemen Akun (Tendik)

Halaman detail manajemen akun digunakan oleh tenaga kependidikan untuk melihat informasi detail akun mahasiswa pada sistem. Pada halaman ini tendik dapat melihat identitas mahasiswa seperti nama, NPM, nomor telepon, alamat domisili, serta foto profil mahasiswa.



Gambar 4.42 Halaman detail manajemen akun

Gambar 4.42 menampilkan tampilan halaman detail manajemen akun. Sistem menampilkan informasi lengkap mahasiswa berdasarkan akun yang dipilih oleh tenaga kependidikan.

Sistem menggunakan metode SWR untuk melakukan pengambilan data secara otomatis dan memperbarui data ketika halaman kembali aktif. Potongan kode berikut digunakan untuk melakukan *fetch* data detail mahasiswa secara *real-time*.

```
const {
  data: student,
  isLoading
} = useSWR(
  studentId
  ? `student_detail_${studentId}`
  : null,
  () =>
    fetchStudentDetailData(
      studentId
    ),
  {
    revalidateOnFocus:
      true,
    refreshInterval:
      60000,
    dedupingInterval:
      5000
  }
);
```

Sistem mengambil parameter id mahasiswa dari URL untuk menentukan data mahasiswa yang akan ditampilkan. Potongan kode berikut digunakan untuk memperoleh id mahasiswa dari parameter halaman.

```
const searchParams =
  useSearchParams();
const studentId =
  searchParams.get("id");
```

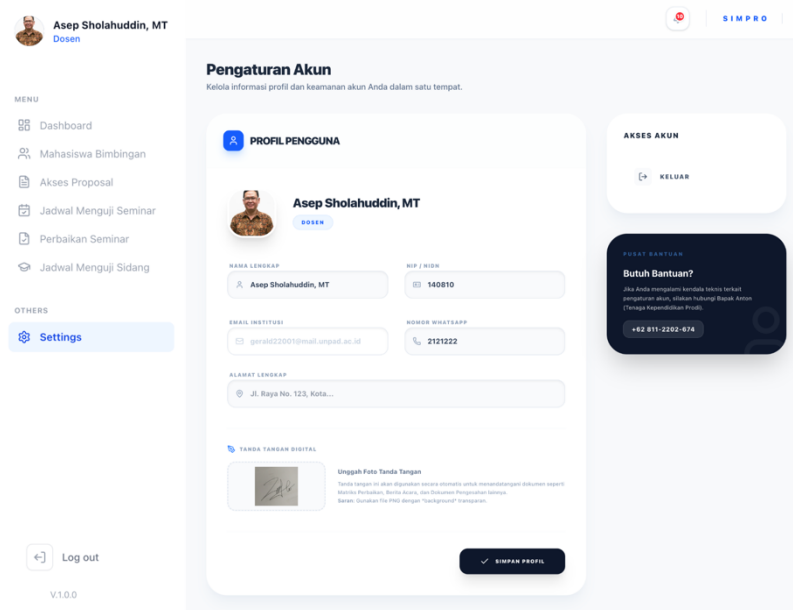
Sistem mengambil data detail mahasiswa dari database berdasarkan id mahasiswa yang dipilih. Potongan kode berikut digunakan untuk memperoleh data mahasiswa dari tabel *profiles*.

```
const {
  data,
  error
} = await supabase
  .from("profiles")
  .select(`
    nama,
    npm,
    avatar_url,
    phone,
    alamat
  `)
  .eq(
    "id",
    studentId
```

```
)  
.single();
```

4.2.43 Halaman Pengaturan

Halaman pengaturan akun digunakan oleh pengguna untuk mengelola informasi profil akun pada sistem SIMPRO. Halaman ini dapat diakses oleh mahasiswa, dosen, kaprodi, maupun tenaga kependidikan. Khusus untuk dosen dan kaprodi, sistem menyediakan fitur tambahan berupa unggah tanda tangan digital yang digunakan untuk proses penandatanganan dokumen akademik.



Gambar 4.43 Halaman pengaturan dosen

Gambar 4.43 menampilkan tampilan halaman pengaturan akun. Sistem menampilkan data profil pengguna seperti nama, email, nomor telepon, alamat, foto profil, serta fitur unggah tanda tangan digital untuk dosen dan kaprodi.

Sistem menggunakan metode SWR untuk melakukan pengambilan data profil secara otomatis serta memperbarui data ketika terjadi perubahan. Potongan kode berikut digunakan untuk memuat data profil pengguna.

```
const {
  data,
  error,
  isLoading,
  mutate
} = useSWR(
  "settings_user_profile",
  fetchProfileData,
  {
    revalidateOnFocus:
      false
  }
);
```

Sistem mengambil data profil pengguna yang sedang login menggunakan autentikasi Supabase. Potongan kode berikut digunakan untuk memperoleh data akun pengguna dari *database*.

```
const {
  data: { user }
} = await supabase
  .auth
  .getUser();
const {
  data,
  error
} = await supabase
  .from("profiles")
  .select("*")
  .eq("id", user.id)
  .single();
```

Sistem memungkinkan pengguna mengganti foto profil akun. Potongan kode berikut digunakan untuk mengunggah foto profil ke Supabase *Storage*.

```
const {
  error: uploadError
} = await supabase
  .storage
  .from("avatars")
  .upload(
    filePath,
    file,
    { upsert: true }
  );
```

Sistem menyimpan URL foto profil ke *database* setelah proses unggah berhasil dilakukan. Kode berikut digunakan untuk memperbarui data foto profil pengguna.

```
await supabase
  .from("profiles")
  .update({
    avatar_url:
      publicUrl
  })
  .eq("id", userId);
```

Sistem menyediakan fitur unggah tanda tangan digital khusus bagi dosen dan kaprodi. Tanda tangan digital digunakan untuk proses penandatanganan dokumen akademik secara otomatis. Potongan kode berikut digunakan untuk mengunggah tanda tangan digital ke storage.

```
const {
  error: uploadError
} = await supabase
  .storage
  .from("signatures")
  .upload(
    fileName,
    file,
    { upsert: true }
  );
```

Sistem memungkinkan pengguna memperbarui informasi profil seperti nama, nomor telepon, NPM/NIP, dan alamat. Potongan kode berikut digunakan untuk menyimpan perubahan data profil pengguna.

```
const {
  error
} = await supabase
  .from("profiles")
  .update(payload)
  .eq("id", userId);
```

Sistem menyediakan fitur keluar akun untuk mengakhiri sesi *login* pengguna. Potongan kode berikut digunakan untuk melakukan proses *logout* akun pengguna.

```
await supabase
  .auth
  .signOut();
router.push("/login");
```

4.3 Pengujian Sistem

Pengujian sistem pada Sistem Informasi Manajemen Sidang Skripsi (SIMPRO) dilakukan untuk mengetahui tingkat kemudahan penggunaan sistem dari sudut pandang pengguna. Metode pengujian yang digunakan adalah *System Usability Scale* (SUS), yaitu metode pengujian *usability* yang bersifat kuantitatif dan terstandarisasi. Pengujian ini bertujuan untuk mengukur sejauh mana sistem dapat digunakan secara efektif, efisien, dan nyaman oleh pengguna sesuai dengan kebutuhan fungsional yang telah dirancang.

Metode SUS dipilih karena mampu memberikan gambaran tingkat *usability* sistem secara sederhana, cepat, dan dapat diinterpretasikan dengan jelas. Pengujian dilakukan setelah seluruh fitur utama sistem selesai diimplementasikan dan dapat digunakan oleh pengguna sesuai dengan perannya masing-masing.

4.3.1 Metode Pengujian *System Usability Scale*

Pengujian dilakukan dengan melibatkan pengguna yang mewakili setiap peran dalam sistem, yaitu mahasiswa, dosen pembimbing, ketua program studi, dan tenaga kependidikan. Setiap responden diminta untuk mencoba fitur-fitur utama sistem, seperti pengelolaan proposal, bimbingan skripsi, unggah dan verifikasi dokumen, penjadwalan seminar dan sidang, penilaian seminar dan sidang hingga verifikasi kelulusan. Setiap pengguna akan diberikan kuesioner yang terdiri dari 10 pernyataan. Kuesioner ini digunakan untuk mengukur persepsi pengguna terhadap kemudahan penggunaan, konsistensi sistem, dan tingkat kepercayaan pengguna dalam mengoperasikan sistem. Skor dari setiap pernyataan kemudian diolah untuk memperoleh nilai akhir SUS.

4.3.2 Data Hasil Kuesioner

Data yang diperoleh dari pengisian kuesioner oleh delapan orang responden direkapitulasi ke dalam sebuah tabel. Kolom P1 hingga P10 merepresentasikan jawaban dari pernyataan pertama hingga pernyataan kesepuluh pada kuesioner SUS dengan skala 1 hingga 5. Hasil rekapitulasi jawaban responden dapat dilihat pada Tabel 4.1.

Tabel 4.1 Data Hasil Jawaban Kuesioner SUS

No	Nama Responden	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
1	Anton	4	2	4	2	4	2	4	2	4	4
2	Rudi Rosadi	5	1	2	2	5	1	5	1	5	4
3	Dimas Falah	4	2	5	2	4	2	4	2	4	2
4	Stevanus Felixiano	4	2	4	1	5	2	4	2	4	2
5	Daffa Burane Nugraha	5	1	4	1	4	2	4	2	4	2
6	Louis Koni Sung	4	1	5	2	4	2	4	1	4	3
7	Muhammad Danendra Syah Hidayatullah	5	1	5	1	5	2	4	1	5	3
8	Akmal	3	2	3	4	3	3	4	3	4	4

4.3.3 Pengolahan Data Kuesioner

Data yang diperoleh dari pengisian kuesioner kemudian dihitung berdasarkan rumus System Usability Scale (SUS) sebagai berikut:

- Untuk pernyataan ganjil: skor = nilai jawaban – 1
- Untuk pernyataan genap: skor = 5 – nilai jawaban

Selanjutnya, total skor dari masing-masing responden dikalikan dengan 2,5 untuk memperoleh skor akhir dalam rentang 0 hingga 100. Hasil perhitungan SUS dapat dilihat dari Tabel 4.2

Tabel 4.2 Hasil Perhitungan SUS

No	Nama Responden	Total Skor Positif (Ganjil)	Total Skor Negatif (Genap)	Jumlah (Positif + Negatif)	Skor Akhir SUS (Jumlah x 2,5)
1	Anton	15	13	28	70
2	Rudi Rosadi	17	16	33	82,5
3	Dimas Falah	16	15	31	77,5
4	Stevanus Felixiano	16	15	31	77,5
5	Daffa Burane	16	17	33	82,5
6	Louis Koni Sung	16	16	31	77,5
7	Muhammad Danendra	19	17	38	95
8	Akmal	12	9	21	52,5
Total keseluruhan skor akhir			615		
Rata-rata skor SUS			76,875		

4.3.4 Analisis Skor SUS

Berdasarkan hasil perhitungan terhadap 8 responden, diperoleh rata-rata skor SUS sebesar 76,875. Nilai tersebut termasuk dalam kategori *good* dan berada pada tingkat *acceptable*, sehingga dapat disimpulkan bahwa sistem yang dikembangkan memiliki tingkat *usability* yang baik serta dapat digunakan dengan cukup mudah oleh pengguna.